Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Kubernetes Networking: Comparative Insights into API Gateways and Service Mesh Implementations

Master's thesis

March 8, 2024

Sami Kuikka

Aalto-yliopisto
Perustieteiden korkeakoulu
Master's Programme in Computer, Communication and Information Sciences

MAISTERINTUTKIELMA
TIIVISTELMÄ

Tekijä:	Sami Kuikka
Työn nimi:	Kubernetes-verkotus: Vertailevia näkemyksiä API-yhdyskäytävistä ja palveluverkkojen toteutuksista
Päiväys:	8. Maaliskuuta 2024
Sivumäärä:	92
Pääaine:	Computer, Communication and Information Sciences
Koodi:	SCI3027.A
Vastuuopettaja:	Professori Petri Vuorimaa
Työn ohjaaja(t):	Juhana Rajala

(finnish)

Mikropalveluarkkitehtuurien yleistyminen ja konttiorkestrointialusta Kubernetesin laajamittainen käyttöönotto ovat luoneet suuren tarpeen verkkoratkaisuille, jotka kykenevät hallitsemaan poikkitoiminnallisia prosesseja erittäin dynaamisissa ympäristöissä. Vaikka Kubernetesin verkkoratkaisut, kuten palveluverkot ja API-yhdyskäytävät, ovat kehittyneet merkittävästi, akateeminen kirjallisuus kärsii vertailevien analyysien puutteesta näiden teknologioiden osalta. Tämä maisterintyö pyrkii täyttämään tämän aukon vertailemalla API-yhdyskäytäviä ja palveluverkkoja Kubernetes-ympäristössä. Työssä tarkastellaan nykyisten verkkoratkaisujen maisemaa sekä arvioidaan niiden vahvuuksia ja heikkouksia, tutkien sekä palveluiden välisiä että asiakas-palvelinvuorovaikutuksia.

Tämän tutkimuksen tulokset osoittavat merkittävää siirtymää teollisuudessa kohti modulaarisia verkkoratkaisuja ja toiminnallisuuksien yhteen sulautumista API-yhdyskäytävien ja palveluverkkojen välillä. Työ korostaa näiden ratkaisujen välisiä eroja ja niiden ymmärtämisen tärkeyttä organisaatioille verkkoratkaisua valittaessa. Vaikka tutkimus keskittyy vertailevaan analyysiin, se on rajoittunut erityisesti Kubernetes- ja pilvipohjaisiin ympäristöihin. Erikoistuminen tekee tutkimuksesta arvokkaan resurssin alalla toimiville organisaatioille, tarjoten tietoa Kubernetes-verkkoratkaisujen hallinnasta monimutkaisessa maisemassa.

Avainsanat:	API-yhdyskäytävä, palveluverkot, Kubernetes-verkotus
Kieli:	Suomi

Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

ABSTRACT OF
MASTER'S THESIS

Author:	Sami Kuikka
Title of thesis:	Kubernetes Networking: Comparative Insights into API Gateways and Service Mesh Implementations
Date:	March 8, 2024
Pages:	92
Major:	Computer Science
Code:	SCI3027.A
Supervisor:	Professor Petri Vuorimaa
Instructor:	Juhana Rajala

(english)

The rise of microservice architectures and the widespread adoption of the container orchestration platform Kubernetes have necessitated networking solutions capable of efficiently managing cross-service functionalities in highly dynamic environments. Despite significant advancements in Kubernetes networking solutions, especially in service meshes and API gateways, there is a notable gap in academic literature regarding comparative analyses of these technologies. This thesis bridges this gap by comparing API gateways and service meshes within a Kubernetes environment, aiming to provide a comprehensive view of the current landscape of networking solutions along with their relative strengths and weaknesses. This involves an examination of service-to-service and client-to-service solutions, evaluating their underlying architectures and offering comparisons between each.

The findings of this thesis highlight a significant shift in the industry toward commoditized networking solutions, highlighting an increasing convergence of functionality between API gateways and service meshes. Crucially, it identifies and discusses the distinctions between these solutions, emphasizing the importance for organizations to understand these differences when selecting their networking solution.

While the thesis offers an in-depth comparative analysis, it is important to note its focus is limited to Kubernetes and cloud-native contexts. This specificity makes it a valuable guide for organizations operating within this domain, equipping them with essential knowledge to navigate the complex landscape of Kubernetes networking solutions effectively.

Keywords:	API Gateway, Service Mesh, Kubernetes Networking
Language:	English

Contents

1 Introduction

1.1 Cloud Native Networking

Over the last decade, application development and architecture have undergone a fundamental shift. Monolithic applications have increasingly given way to microservice architectures, while containerized environments are replacing traditional bare metal and virtual machine setups. There has also been a noticeable migration from on-premise hosting to cloud-native platforms (Li et al., 2019).

As applications and their architectures evolve, so too do the underlying technologies integral to their operation. One such technology is the API Gateway, which serves as a pivotal component in application infrastructure, unifying common features across various services under a single facade (De and De, 2017). API gateways are designed to fulfill a range of organizational goals within applications. They offer an extensive feature set that typically includes authentication, authorization, load balancing, and caching, among others (Zhao et al., 2018). Although the features provided by API gateways are largely consistent across different providers, their main objective is to consolidate similar functionalities from different services into a single, client-facing interface (Gamez-Diaz et al., 2019).

As application architectures have evolved toward a microservices model, API gateways have remained a crucial component. Their use has proven to offer numerous benefits, which establishes them as essential within microservice architectures (Zhao et al., 2018). However, as the foundational elements of application development have shifted, from monolithic to microservice architectures, API gateways have also evolved to meet the demands of modern applications. I will refer to these evolved API gateways, designed specifically for cloud-native architectures (i.e., microservices), as "Cloud Native API Gateways" to emphasize that their purpose is to integrate seamlessly with the needs of cloud-native applications.

There are multiple reasons for the rise of Cloud Native-focused API Gateway solutions. One primary driver is the increasing adoption of microservice architectures within organizations, drawn by inherent benefits such as built-in scalability (Balalaie et al., 2016). Furthermore, the role of APIs in modern applications cannot be overstated. There has been a sustained trend towards the growing demand for providing external users with access to data through RESTful APIs (De and De, 2017; Harms et al., 2017; Neumann et al., 2018). As a result, RESTful APIs have emerged as the accepted norm for contemporary application interfaces in the industry (Schermann et al., 2016). APIs have essentially become commoditized, leading to a noticeable shift towards an API economy (Tan et al., 2016). The increasing importance of APIs inevitably means that the methods

of accessing them have also gained significance.

As previously discussed, API gateways act as a facade between clients and services, managing client-to-service communication, also known as north/south traffic. They actively serve as a unified entry point that clients use to access APIs (Montesi and Weber, 2016). In practice, they perform various operations on requests from clients to provide features such as authentication and load balancing (Zhao et al., 2018).

The highly dynamic nature of Cloud Native applications makes it necessary for API gateways to be compatible with microservice architectures. The fixed nature of traditional API gateways in the infrastructure was not ideal for Cloud Native environments, often leading to interoperability challenges and a lack of standardized modeling for API gateways (Gamez-Diaz et al., 2019). However, the shift towards microservice architecture has caused a movement toward better interoperability and standardization among API gateway providers to address issues inherent in Cloud Native architectures (Gamez-Diaz et al., 2019).

At the beginning of this section, I referenced a pivotal shift toward containerized applications. This packaging of applications as containers has had significant implications for software development, particularly in the management of containers, which is inherently challenging. This difficulty in container management gave rise to container orchestration platforms like Kubernetes (Bernstein, 2014). Kubernetes now plays a central role in Cloud Native applications, having seen rapid adoption among organizations developing Cloud Native solutions (Miyachi, 2021; Cloud Native Computing Foundation, 2022).

The rise of Kubernetes implies that Cloud Native API Gateways must be easily integrated to work with container orchestration platforms. Kubernetes opens new possibilities for achieving the same goals as API gateways, with service mesh solutions emerging as the infrastructure layer that handles service-to-service (east/west) communication, thus overlapping with API gateway features (Li et al., 2019). These solutions offer features that overlap with those of traditional API gateways, thus there is no one-size-fits-all solution when choosing the right solution for Kubernetes networking. However, solutions such as service meshes also have their own challenges and pursue different goals than traditional API gateways do (Li et al., 2019).

This Master's thesis aims to provide the reader with a comprehensive view of Kubernetes networking by considering alternative solutions that address the same organizational objectives. While the objectives, including unified authentication, authorization, and load balancing, have stayed the same, the growth of the cloud-native ecosystem has brought a wider array of tools to achieve these aims. Therefore, this thesis will explore various approaches designed to tackle the inherent challenges of networking. Additionally,

it will demonstrate the progression of API gateways within the cloud-native space and guide practitioners in evaluating their options for achieving their Kubernetes networking goals.

1.2 Company Context and Motivation

Networking tools are never the goal but rather the means to achieve organizational objectives. The decision to choose the right tool must be evaluated in the context of the organization. Within this framework, I will introduce Vertex Systems, a Finnish company specializing in Computer-Aided Design (CAD) and Product Lifecycle Management (PLM) software, which serves as a case study for this Master's thesis. This section aims to provide an overview of Vertex Systems to facilitate the evaluation of different approaches through a real-world example later in this thesis.

1.2.1 Company Profile

Vertex Systems, a Finnish CAD software company, has been creating CAD software since its establishment in 1977. With a global footprint, Vertex's software is utilized in over 100 countries worldwide. The company has a diverse portfolio of CAD products, ranging from mechanical engineering solutions to specialized tools for plant and pipeline design. Vertex Systems has acknowledged the significance of APIs and, as a result, is developing a new set of products. These products aim to make CAD data accessible via APIs, allowing users to access project data more flexibly from anywhere.

Vertex Systems' adoption of cloud-native patterns, evident in the architecture and workflow of their new product suite, has been at the core of its shift towards accessible APIs. The company has embraced a microservice architecture for services orchestrated using Kubernetes. This decentralized approach allows for greater autonomy among development teams, who can manage the full life-cycle of their services independently.

Due to its global user base, Vertex implements a detailed deployment strategy for its products. Each product is hosted on its own Kubernetes cluster for every geographical region in which it operates. Additionally, different environments, such as testing and production, each have their dedicated clusters. Consequently, the new product Vertex Sync, which aims to make CAD data accessible, will be supported by multiple clusters even within the same geographical area—for example, "sync-test-westeu" and "sync-prod-westeu" clusters.

Furthermore, it is crucial to note that these clusters, despite being distinct, are interconnected. While the products differ, there can still be essential communication between services across different clusters (see Figure 1). Additionally, Vertex is committed

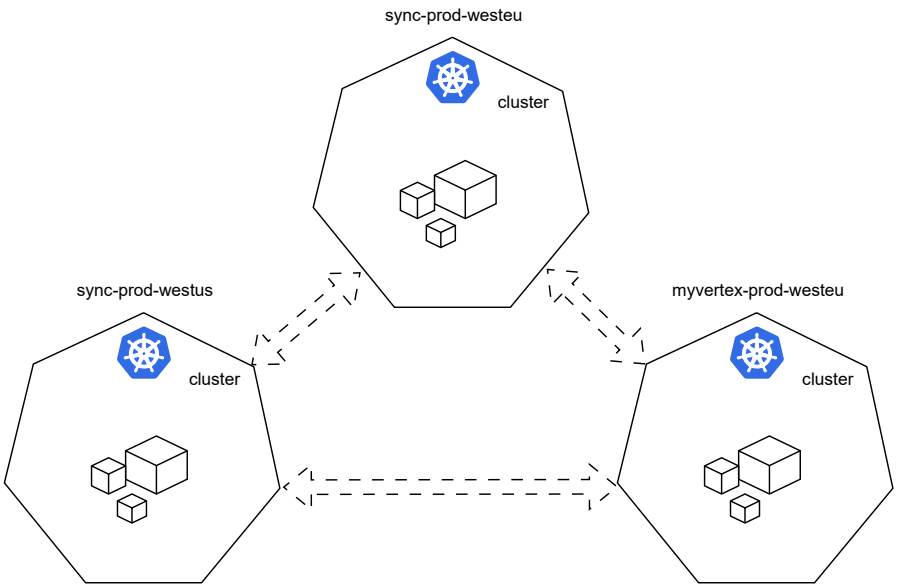


Figure 1: Vertex products have their own Kubernetes clusters which communicate with each other

to cloud-vendor-neutral solutions due to the need to work with various regional cloud providers. A prime example of this is in China, where a local cloud provider is required.

Vertex is actively pursuing an efficient and secure Kubernetes networking solution. Consequently, the company is exploring different approaches to Kubernetes communication. This exploration offers this thesis a robust case study on how various approaches to Kubernetes networking meet the needs of complex environments like Vertex Systems.

1.3 Research Objectives and Questions

This section outlines the main objectives driving this research, the significance of the study in the broader context of the cloud-native landscape, and the key questions I aim to answer through this Master's thesis.

1.3.1 Research Objectives

Although API gateways are a well-researched topic with many popular solutions existing (Ali and Zafar, 2022), there are gaps in the academic literature regarding Cloud Native API gateway solutions and their overall role in cloud-native networking. Given the rise of cloud-native applications, there is a need for versatile solutions to manage both north/south and east/west traffic. This research aims to address several objectives:

1. **Historical Analysis of API Gateway Evolution**: One of the goals of this research is to understand how API gateway technologies have evolved with cloud-native

applications and to compare these new Cloud Native API gateways with their predecessors.

2. **Examination of Current Approaches**: Secondly, I will investigate the various approaches to managing traffic in a cloud native Kubernetes environment, aiming to identify the benefits and challenges of each approach.

3. **Comprehensive Overview for Practitioners**: Finally, I aim to provide a holistic view of Kubernetes networking, offering practitioners an informed perspective that enables them to choose the right tools for their Kubernetes networking objectives.

From these research objectives, we can discern the significance of the study. This study bridges the academic-practical gap by providing theoretical insights and empirical evaluations of networking solutions in cloud-native environments. It also can empower decision-makers to base their decisions on Kubernetes networking solutions that adhere to industry best practices. In summary, this research aims to benefit both the academic community and industry practitioners in the areas of Cloud Native API Gateways and the broader concept of Kubernetes networking.

1.3.2 Research Questions

Within this context, this research seeks to address the following research questions:

Research Question 1: *How have API Gateway architectures evolved over the past decade?*

This research question delves into the historical progression of API Gateway architectures. It aims to provide a comprehensive understanding of the transformations that have occurred, highlighting key advancements and their implications for microservice architecture.

Research Question 2: *What are the prevalent contemporary solutions for service networking in cloud-native architectures?*

This question focuses on the current state of north/south and east/west traffic in cloud-native environments. By evaluating the available options, this research aims to offer insights into the most effective strategies for ensuring secure, efficient, and reliable communication in modern cloud-native applications.

Research Question 3: *What are the relative strengths and limitations of API gateways, Ingress controllers, and Service meshes in managing Kubernetes networking?*

This research examines various approaches to Kubernetes networking for both north/south and east/west traffic. Based on this analysis, I will provide a comprehensive view of how these methods compare, enabling practitioners to select the best solution for their organization's objectives. In addressing these questions, the research aims to provide a

holistic and well-rounded perspective on Cloud Native API Gateways, comparing them to other methods that strive to achieve similar objectives.

1.4 Thesis Structure

This thesis is organized into sections to systematically address the complexities of Kubernetes networking. Section 2 starts by introducing essential background concepts. It starts with Section 2.1, highlighting the growing role of APIs. This leads to Section 2.2 about API management. Next, Section 2.3 discusses the history and development of API gateways. Section 2.4 looks at cloud-native applications and how they're changing software design. Section 2.5 focuses on Kubernetes and its importance in cloud-native environments. The section ends with Section 2.6, tying together cloud-native ideas, Kubernetes, and API gateways.

Section 3 dives into a detailed literature review. The discussion begins with Section 3.1, examining the evolution of API gateway architecture. Section 3.2 expands the discussion to include API gateways, Ingress controllers, and service meshes within Kubernetes.

Section 4 presents a thorough comparison of Kubernetes networking solutions. It starts with Section 4.1, comparing solutions based on their architecture. Section 4.2 contrasts different kinds of solutions and their features. The section concludes with Section 4.3, looking at how standardization is affecting Kubernetes networking.

Section 5 concludes the thesis by summarizing the findings. Section 5.1 provides a summary of the main findings. Section 5.2 explores how these findings impact organizations and strategy. Section 5.3 looks at the wider implications for Kubernetes networking. Section 5.4 applies the findings to Vertex Systems specifically. Section 5.5 talks about the research's limitations and suggests future research areas. Finally, Section 5.6 wraps up the thesis, highlighting its key contributions to the field of Kubernetes networking.

2 Background

To understand the significance of API gateways and their impact on cloud-native applications, it is important to first comprehend the overall rationale for using API gateways in software development. This section will begin with the emergence of APIs and from there construct a foundation of knowledge leading up to Cloud Native API Gateways which is used as a starting point for discussion on Kubernetes networking solutions.

2.1 The API Revolution

Central to cloud-native API gateways are the Application Programming Interfaces (APIs) themselves. APIs are the foundation upon which modern software architectures are built. They establish an agreement for a software component by specifying the protocol, data format, and endpoint, thereby facilitating communication between two applications over a network (Brajesh, 2017).

The importance of APIs becomes even more highlighted when we discuss dynamic environments such as cloud-native applications. In such environments, APIs facilitate seamless interaction between disparate systems, allowing them to understand and communicate with each other (Brajesh, 2017). The key functionality of APIs is to facilitate this communication without letting various technologies, languages, and platforms disrupt the interaction.

APIs can be classified into three types based on their end users:

1. **Public APIs** are accessible to all users. They expose the API to a wide range of users.
2. **Private APIs** are exclusively used for internal application integrations. They are accessible only to an organization's internal users.
3. **Partner APIs** are internal APIs that are also exposed to specific users outside of the organization. These APIs are typically shared with trusted partners or affiliates. For instance, B2B Partner Integration APIs fall under the category of partner APIs.

An important aspect of APIs is that they unlock business assets for different user groups. For example, private APIs make business assets accessible within an organization. APIs offer a controlled way to access core functionalities, data, and services. They enable internal collaboration within organizations, while public APIs allow external users to access data, thus creating new business opportunities.

As organizations grow, so does the volume of APIs in their ecosystem. This creates the need to handle the API expansion appropriately. This involves not only creating APIs efficiently but also ensuring their reusability, establishing robust API governance practices, and utilizing existing APIs to their fullest (Mulesoft, 2019). One way to manage this API expansion is through API management solutions.

2.2 API Management

As organizations evolve, the number of APIs within their ecosystems increases. The increase in API endpoints implies that their effective management becomes important to harness the full potential of APIs.

API Management is an activity that enables organizations to design, publish, and deploy their APIs for (external) developers to consume (Mathijssen et al., 2020). It includes a range of activities aimed at orchestrating the lifecycle of APIs, from design and publication to deployment and consumption. This practice enables organizations to expose their assets to external users who seek to leverage these APIs.

API management platforms are a key component of API management, acting as platforms where organizations can publish their APIs, thereby unlocking their assets for different user groups within the organization. These platforms provide core capabilities important for API programs, including developer engagement, business insights, analytics, security, and protection (Brajesh, 2017).

API management platforms offer a range of key capabilities, including publishing APIs, controlling access, enabling authentication, managing data flows and API lifecycle, ensuring security, and providing robust analytics and monitoring tools. Additionally, they support API design, offer comprehensive documentation, and facilitate usage throttling. These platforms also streamline the deployment of APIs (Mathijssen et al., 2020). The specific capabilities available may vary depending on the API platform provider and the architectural framework of APIs within the organization.

These capabilities can be categorized into four distinct groups (Brajesh, 2017):

1. Developer Enablement for APIs
2. Secure, Reliable, and Flexible Communications
3. API Lifecycle Management
4. API Auditing, Logging, and Analytics

These functionalities are supported by three vital components within the API management platform (Brajesh, 2017):

1. **API Gateway:** The API Gateway serves as a pivotal tool for creating and managing APIs using existing data and services. It goes beyond basic connectivity, allowing the addition of security measures, traffic management, interface translation, orchestration, and routing capabilities.

2. **Developer Portal:** The Developer Portal is a comprehensive platform that caters to developer and app registration and onboarding. It offers rich API documentation, robust community management features, and can support API monetization.

3. **Analytics Service:** This service plays a critical role in tracking and analyzing traffic generated by individual applications. It provides metrics for organizations about the usage of APIs and their performance.

2.3 API Gateways

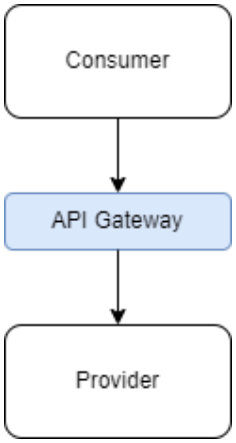


Figure 2: API Gateway guards the boundary of the application

API Gateways acts as a bridge, facilitating secure, flexible, and reliable communication between the boundary of consumer and provider (Figure 2). API Gateways are responsible for client-to-service communication, known as north/south traffic. They serve as a boundary between the client and the service, controlling how the services are accessed externally and thus seen as facade in front of the backend services (Brajesh, 2017).

In the context of the API gateway pattern, it's noted that there's an additional network hop that each request must traverse to access the underlying APIs. This aspect is sometimes referred to as a centralized deployment (Reisz, 2022).

2.3.1 API Gateway Features

The features offered by an API gateway can vary significantly depending on the provider and the specific implementation. Drawing on (Brajesh, 2017), this section highlights the

potential functionalities of API gateways, illustrating their role in Kubernetes networking.

Core functionalities:

- **Security**: API gateways implement unified security measures across services, including authentication, authorization, data encryption, and protection against threats such as DDoS attacks and unauthorized access. This ensures secure data transmission.
- **Traffic Management**: API gateways ensure smooth service traffic flow by employing strategies such as consumption quotas, spike arrests, usage throttling, and traffic prioritization to maintain service availability.
- **Interface Translation**: API gateways facilitate seamless communication between different systems by translating between various data formats.
- **Caching**: Some API gateways are capable of caching responses to enhance response times and reduce backend load.
- **Routing**: API gateways direct incoming requests to the appropriate services, utilizing rules for traffic direction and load balancing.
- **Service Orchestration**: API gateways can coordinate complex sequences of service-to-service requests, ensuring that multi-step processes are executed in the correct order.
- **API Analysis**: Through monitoring and analytics, API gateways provide insights into API usage patterns, aiding in an organization's strategic decision-making.

In summary, API gateways are tools that enhance the overall application architecture by providing unified features to the infrastructure, such as security and routing. This helps manage the complexity of modern API ecosystems, making them indispensable in the management of modern application infrastructures.

2.4 Cloud Native

Before delving into the specifics of Cloud Native API Gateways, it is crucial to understand the infrastructure they are built upon. At its core, "cloud native" is not just about running applications on the cloud; it represents a paradigm shift in how applications are built, deployed, and managed. The Cloud Native Computing Foundation (CNCF) describes cloud native technologies as tools that "empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds."

(Cloud Native Computing Foundation, 2023a). Although no precise definition exists for cloud-native applications, Kratzke and Quint (2017) define them as applications that adhere to principles including operating on automated platforms, utilizing software-based infrastructure and networks, and possessing the ability to migrate and interoperate across different cloud infrastructures. Cloud Native applications are complex and can consist of many different elements. At their heart are services, designed using a microservice architecture (Gannon et al., 2017). These services are operated within containers, which are managed by their own technologies. Adopting cloud-native practices aims to optimize resources, enhance scalability, improve resilience, and increase deployment flexibility.

It's crucial to note, however, that this shift towards cloud-native wasn't arbitrary. As the digital landscape evolved, so did the demands and requirements for modern applications. Increasing complexity and its inherent problems started the emergence of cloud-native architectures.

2.4.1 Evolution of Application Requirements

In recent years, the digital landscape has experienced rapid shifts in both technology and user expectations. These changes have led to a significant evolution in application requirements, which, in turn, have caused application architectures to change. Modern applications necessitate specific capabilities, such as achieving zero downtime, shortening feedback loops, supporting multiple devices, and being data-driven, as outlined by Davis (2019).

The first principle, **Zero Downtime**, stems from new user requirements. Applications are expected to operate with minimal downtime. Errors in the application or updates to the services should not cause interruptions to the application itself. Applications should be designed in a way that supports the principle of zero downtime. Achieving this requires constructing the software architecture with loosely coupled components. Using loosely coupled components means that errors in one component have minimal disruption to others. Even brief application unavailability can lead to revenue losses and potentially damage the organization's reputation. Therefore, modern application architectures are designed to minimize service disruptions.

The second noticeable shift in modern application development is the **Shortened Feedback Loops**. Developers have moved from spreading code releases over lengthy intervals to adopting rapid, frequent releases. This agility gives businesses a competitive advantage, enabling them to receive feedback from the market faster. This shift is also one of the reasons for the rise of microservice architectures because microservice architecture allows different services to be developed independently thus allowing faster deployments.

The third new requirement for applications is the emphasis on **Multi-Device Support**.

Today, a diverse array of devices, including smartphones and smart wearables, must be able to access the same applications. This demand is not just about visual or functional consistency across different devices but also about how different devices interact with the data. Modern applications should take these device-specific data interactions into account.

Finally, modern applications are unmistakably **Data-Driven**. Data now resides more widely distributed across environments, from on-premises to cloud-based storage. Moreover, modern applications now handle significantly larger volumes of data. Consequently, contemporary applications often manage a large amount of data that is stored in a distributed manner.

In summary, modern applications have many new requirements that need to be addressed. This situation underscores the critical role of cloud-native applications in striving to meet these evolving challenges.

2.4.2 Rise of Modular Design in Modern Application Architecture

The last section described the new requirements expected from modern applications. However, these evolving requirements are not merely challenges; they also present opportunities to improve application architecture within organizations. Central to this architectural transformation are modular design principles. This section explains why modular design has emerged as the one of the solutions to address the new demands of applications.

Minimized Failure Radius: One of the advantages of modular design is its inherent resilience against system-wide failures. In a traditional monolithic application, a failure in one component can spread across the entire application. In contrast, errors in a modular design are contained within the loosely coupled component where the error occurred. Thus, while a disruption in one component may affect certain functionalities, the modular design ensures the rest of the application remains operational. Consequently, modular design is an excellent choice to ensure zero downtime in applications because disruptions have minimal impact in a modular setting.

Facilitated Release Cycles: As a monolithic application grows in complexity, it inevitably creates numerous interdependencies among its various components. These interdependencies slow down software updates, as changes to one part of the code can impact other areas throughout the application. By dividing functionalities into discrete modules, coordination and dependency issues can be significantly reduced. Each module can be developed, tested, and deployed independently. This modularity can accelerate development progress, and as such, modular design also helps to meet the demand for shorter feedback loops.

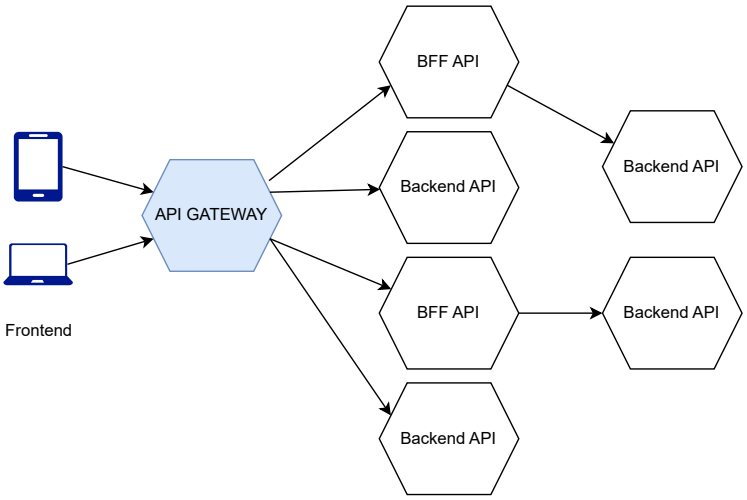


Figure 3: Example microservice architecture

Dynamic Scalability: As mentioned in the previous section, the volume of data has increased in modern applications. This data can be unevenly distributed across different components of the application. Modular design offers significant benefits for application scaling. It allows for scaling only the components that require it, making the process more targeted and efficient than in monolithic applications. Moreover, scaling can occur automatically by instantiating new instances of a modular component to accommodate sudden spikes in data.

2.4.3 Cloud-Native Architecture in the Modern Digital Ecosystem

In the preceding section, I underscored the evolving landscape of application requirements and how modular design addresses these new demands. Now, let's shift our focus to cloud-native architecture, the architectural style that builds upon the principles of modular design.

Davis (2019) describes cloud native as follows: "Cloud-native software is highly distributed, must operate in a constantly changing environment, and is itself constantly changing." This encapsulation captures the dynamic, distributed, and ever-evolving nature of cloud-native applications. But why the emphasis on "cloud" in cloud-native? The cloud isn't just a hosting solution in the context of cloud-native. Cloud-native applications are designed to work seamlessly in cloud environments while leveraging the benefits of it. These benefits include the cloud's scalability and distribution capabilities, as well as its automation features for deployments and self-healing mechanisms.

Before delving into the bigger picture of cloud native, it is important to understand the anatomy of cloud-native applications. Typically, a cloud-native application consists of several of these components:

1. **Frontend**: This is the user-facing component, responsible for rendering information and interacting with users. It offers a visual interface, presenting data and functionalities in an accessible manner.

2. **Backend**: The backend components are responsible for specific sets of functionalities. They operate through APIs, offering services that range from data processing to business logic execution.

3. **Backend for Frontend (BFF)**: Acting as an intermediary, the BFF sits between the frontend and the backends. Its primary role is to aggregate and transform data from backend services, streamlining the flow of information to the frontend.

4. **Database**: This foundational component is where data resides. Cloud-native architectures often employ distributed databases, ensuring data availability, redundancy, and resilience.

Now, returning to our broader understanding of the architecture: Often synonymous with microservices architecture, cloud-native software is characterized by its decentralized construction. Cloud-native applications can be visualized as a network of independent, deployable units known as "services." Each of these services handles a certain functionality or subdomain in the overall architecture. These mini-services are focused on implementing narrowly specific functionalities in the system (Richardson, 2018). As a result, cloud-native applications are systems made up of smaller services that together create the whole product.

Building upon our understanding of cloud-native architecture, it becomes evident that cloud-native architecture has many advantages. These advantages include, but are not limited to:

1. **Simplicity in Service Design**: By breaking down applications into smaller, focused services, the complexity inherent to monolithic designs is sidestepped. Each service becomes more straightforward, easier to understand, design, and maintain.

2. **Team Autonomy**: Cloud-native's decentralized approach empowers individual teams to take ownership of their respective services. This autonomy means teams can work independently, and make decisions more rapidly without needing extensive coordination with other teams.

3. **Accelerated Deployments**: The modular nature of microservices, coupled with the automation capabilities of cloud platforms, results in faster code-to-deployment cycles. Small, incremental changes can be pushed to production without the extensive downtime traditionally associated with large-scale updates.

4. **Technological Diversity**: Unlike monolithic architectures that may lock organizations into specific technology stacks, microservices embrace technological diversity. Each service can be developed using the best-fit technology for its requirements, whether it's a specific programming language, framework, or database.

5. **Enhanced Fault Isolation**: In the unlikely event of a service failure, the blast radius is confined to that particular service, ensuring better fault isolation. This containment prevents widespread system outages and ensures system reliability.

6. **Granular Scalability**: Each service can be independently scaled based on its workload and demand, allowing for efficient resource utilization without over-provisioning.

7. **Flexibility in Service Lifecycle Management**: With the microservices approach, individual services can be deployed, upgraded, or even replaced without causing disturbances to the overall application.

While cloud-native architecture offers benefits, it is not devoid of challenges. Addressing these challenges are pivotal to take full advantage of cloud-native design:

1. **Service Discovery**: In a cloud-native environment, the dynamic nature of services creates a unique challenge. As services are dynamically scaled, upgraded, or terminated, there's an essential need to keep track of the current state and location of services. Service discovery tools are important to work within a cloud-native architecture. They allow systems to automatically detect service instances, thus ensuring that services seamlessly interact with the overall system without manual intervention.

2. **Security Concerns**: Distributed microservice architecture has a larger attack surface than monolithic applications, due to each service having its unique endpoint, which can all become potential targets for attackers. Consequently, while microservices can be isolated to prevent system-wide failures, they also create the need for robust security measures. Overall, the security concerns of microservice architecture become complex due to the large number of systems that communicate with each other within a microservice architecture.

3. **Observability and Monitoring**: The autonomy and flexibility that microservices offer also lead to increased system complexity. This complexity translates to challenges in understanding the holistic health and performance of the system. Observability tools become more important in cloud-native applications to understand the communication happening between microservices. Observability should be able

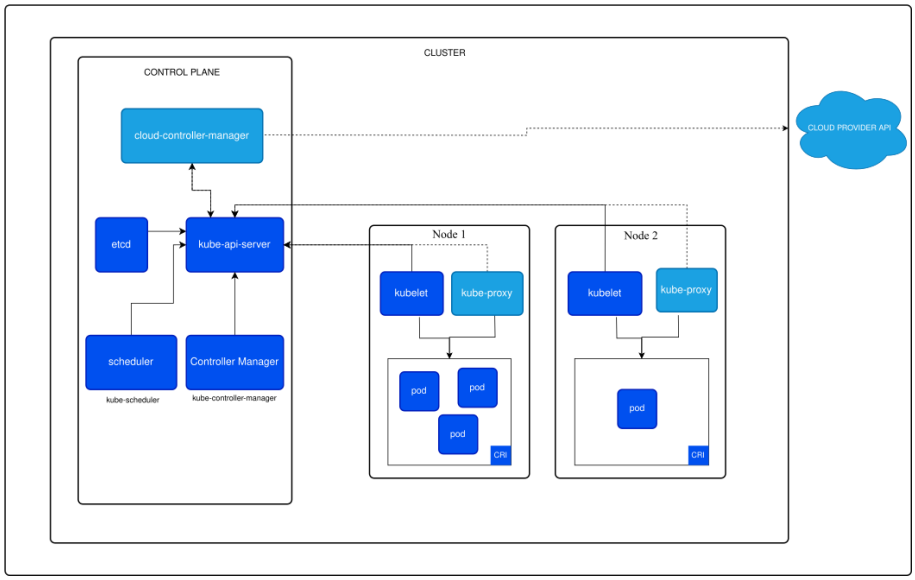


Figure 4: Kubernetes cluster architecture (Kubernetes, 2023a)

to provide both a holistic view of the traffic to the application and also allow for a more granular inspection of the individual services to troubleshoot issues in the communication.

While the journey to cloud-native does present notable challenges, it also highlights the need for tools and platforms that are specifically engineered to address these challenges. One platform that stands out in the cloud-native ecosystem is Kubernetes. In the next section, we will explore the basics of Kubernetes, which is an important technology to know to understand networking in cloud-native systems.

2.5 Kubernetes

One of the fundamental changes in abstraction in software development is the rise of container deployment (Li et al., 2019). Containers are also a foundational concept in understanding Kubernetes. A container image resembles a package, it contains an application, its operating environment, and essential instructions on how it should operate. It can be imagined as a self-contained box with everything an application needs to run without requiring any external dependencies.

A standout feature of containers is their modular design. Just like building blocks, each container is designed for a specific task. When combined, these blocks form a complete and functional structure, emphasizing the benefits of modular design in cloud-native applications.

Another advantage is their efficiency. Unlike virtual machines that need an entire operating system to run an application, containers are much smaller in size. They use

Linux container technology isolates each service, allowing multiple services to operate on the same host without interfering with each other.

By understanding containers, we can shift our focus to Kubernetes, which is a platform for orchestrating these containers. As defined by its official documentation, "Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications." (Kubernetes, 2023d).

Reinforcing its dominance in the cloud-native ecosystem, a report from the Cloud Native Computing Foundation (CNCF) highlights that Kubernetes is used by an overwhelming 89% of the CNCF end users (Cloud Native Computing Foundation, 2022). This statistic further emphasizes the pivotal role Kubernetes plays in the current technological landscape.

2.5.1 Kubernetes Features

A notable feature of Kubernetes is its ability to treat an entire cloud data center as if it were a single deployment platform. This unifies container deployment processes and optimizes resource allocation in the infrastructure. Kubernetes has emerged as the preferred solution for the management of containerized applications, mainly due to its extensive features. Kubernetes offers some of the core features, such as:

1. **Service Orchestration**: Kubernetes manages service coordination and ensures that services operate in harmony.
2. **Self-healing**: Kubernetes continuously monitors its managed containers. In the event of a failure, it can autonomously initiate measures to address the issue, such as launching a new instance of the service.
3. **Automated Rollouts and Backups**: With Kubernetes, you can smoothly transition updates or modifications to the system. If any issues surface, the platform has the capability to revert to a previously stable backup, ensuring system consistency.
4. **Automatic Scaling**: As demands on applications fluctuate, Kubernetes dynamically adjusts resources, ensuring optimal performance and resource efficiency.
5. **Service Discovery**: This feature ensures that new services or components are identified and integrated, allowing for an adaptable and evolving system.
6. **Load Balancing**: Kubernetes ensures the efficient distribution of incoming traffic amongst its managed containers, preventing potential bottlenecks in the servers.

Collectively, these attributes illustrate the capabilities of Kubernetes, further justifying its position as an indispensable tool in the cloud-native domain.

2.5.2 Kubernetes Components

I will briefly introduce the basic components of Kubernetes here, as readers of this thesis should understand the basics of Kubernetes. Later in the thesis, a more in-depth view is given of the relevant Kubernetes concepts. These Kubernetes components collaborate to orchestrate the deployment, management, and scaling of containerized applications. Here is a breakdown of the primary components of Kubernetes (Kubernetes, 2023b):

1. Cluster: At its core, a Kubernetes system is built around the concept of a ”cluster”. Defined as a set of worker machines, these machines are referred to as nodes. The fundamental purpose of a cluster is to run containerized applications. Clusters have at least one worker node.

2. Node: Delving deeper into what constitutes a cluster, we encounter ”nodes”. A node is an individual entity—be it a machine or a Virtual Machine (VM) — that has the primary function of running pods.

3. Control Plane: The container orchestration layer that exposes the API and interfaces to define, deploy, and manage the lifecycle of containers. This layer consists of many different components, such as (but not restricted to) (Kubernetes, 2023b):

1. etcd: Persistent data store storing cluster configuration
2. API Server: The API server is the frontend for the Kubernetes control plane
3. Scheduler: Schedules the applications, assigns worker nodes for deployable components
4. Controller Manager: Executes functions at the cluster level, including component replication, monitoring worker nodes, managing node failures and so on (Luksa, 2017)
5. Cloud Controller Manager: Embeds cloud-specific control logic

These components can be run as traditional operating system services (daemons) or as containers.

4. Pod: Scaling down to even more granular levels, we have the ”Pod”. Considered the most elementary Kubernetes object, a pod represents a set of containers running synchronously on the cluster. Generally configured to execute a singular primary container, pods can also concurrently run supplementary sidecar containers. These sidecar containers can provide additional functionalities, such as logging.

5. Container: A lightweight and portable executable image that contains software and all of its dependencies.

In the subsequent section, we will finally introduce the concept of Cloud Native API Gateways, which is built upon the knowledge gained so far from Cloud Native architecture and API gateways. We will see how dynamic environments such as Kubernetes affect the API gateways and how API gateways adjust to these dynamic settings.

2.6 Cloud Native API Gateways

The shift towards cloud computing and Kubernetes-based orchestration has undeniably revolutionized application deployment and management. However, this change introduces challenges for legacy technologies that need to be addressed to work seamlessly within new, highly dynamic architectures. This section introduces API Gateways that have evolved to become part of these Cloud Native ecosystems, known as Cloud Native API Gateways.

Services in cloud-native platforms, such as Kubernetes, are inherently dynamic. The conventional API gateways were not designed for such dynamic environments where services can scale and change locations. This dynamism means that an agile approach to service discovery and interaction is needed from the API gateway solution.

Further challenges for traditional API gateways arise from the diverse requirements of services. For example, services may use different communication protocols or have various load-balancing strategies. Also, services are more granular in a microservice architecture. API gateway solutions should handle these diverse requirements and still ensure a cohesive interface for the end-users.

Moreover, the rapid increase of services in cloud-native designs justifies the need for a centralized control plane. Such a structured approach ensures efficient orchestration and maintains the system's coherence.

In cloud-native applications, with the adoption of a full-cycle development approach, developers' roles have expanded to cover all aspects of the application lifecycle. Given the increased complexity of managing service interactions in such an environment, API Gateways are crucial, acting as a central point to ensure seamless communication between services.

Following the exploration of the fundamental considerations for API gateways in cloud-native architectures, it's crucial to understand why traditional or "legacy" API gateways face challenges in cloud-native environments. The main reason is the dynamic nature of cloud-native applications. Traditional API gateways were introduced in an era when the infrastructure was more static and predictable.

Drawing insights from (Gunatilaka, 2023), it becomes evident that many traditional API gateways were deeply entwined with specific infrastructure components and technologies.

In the cloud-native landscape, where agility and adaptability are essential, these dependencies are limiting. As aptly highlighted, these dependencies prevent the "ability to use the benefits of cloud platforms, including auto-scaling, infrastructure as code practices, and multi-cloud deployment" (Gunatilaka, 2023). As there are multiple cloud providers, cloud-native architecture underscores the importance of using technology and provider-agnostic tools, thereby leveraging the advantages of different platforms.

Furthermore, the monolithic design that defined many API management solutions did not meet the needs for dynamic scaling that occurs in microservice architecture. The ability to scale services individually is a cornerstone of cloud-native applications. Monolithic API management solutions scale collectively, and thus can't provide the needed granular control that cloud-native applications require. In conclusion, while legacy API gateways proved robust and effective in their original context, the cloud-native domain, with its unique challenges and demands, requires a reimagining of the gateway solution.

Transitioning from the challenges of legacy API gateways, we now delve into the realm of Cloud Native API Gateways. These gateways, specifically designed for the dynamism of cloud-native frameworks, bring about an integrated approach to managing application interfaces.

In Kubernetes and cloud environments, the new-age API Gateway is no longer just a gateway; it combines the roles of a load balancer, an application delivery controller (ADC), and a traditional API gateway, which used to be distinct part of the system (Li, 2020). This integration is a response to the demands of cloud-native applications, offering a unified solution in place of fragmented legacy systems.

Load balancing, previously managed by dedicated components within Kubernetes clusters, now seamlessly falls within the scope of these modern gateways. They also embed traditional API management features, such as authentication, and ADC-like capabilities, including rate-limiting and retries, as highlighted by Li (2020).

A standout feature of Cloud Native API Gateways is their real-time service discovery. Since services in Cloud Native applications may change locations or scale dynamically, real-time service discovery ensures that these services remain accessible, adapting automatically to changes.

2.6.1 Advancements in Cloud Native API Gateways

Continuing our exploration into Cloud Native API Gateways, two paramount shifts in this area need further discussion: the commoditization of API gateways and their inevitable move towards standardization and interoperability. A more in-depth analysis of how these are achieved can be found later in the thesis.

The Commoditization of API Gateways

Historically, legacy API gateways operated as a central part of API management solutions. Thus, the question was not about choosing an API gateway; rather, focused on which API management platform the organization should select. This choice also dictated the other tools that could be used with the API gateway, such as developer portals or cloud vendors.

Today, API gateways are becoming more commoditized. This means that API gateways are not merely part of API management; they are, in a sense, just another interchangeable component within the infrastructure. Historically, this was not possible, as API gateways formed the central pillar upon which all API management capabilities were built. Organizations are no longer restricted to a single gateway provider, enabling them to select a gateway solution that best meets their specific needs. This commoditization shift is particularly evident in a Kubernetes environment, where API gateways can become an integral part of the overall infrastructure. Consequently, vendor-neutral API gateway solutions have emerged that can be utilized within Kubernetes clusters.

The Push for Standardization and Interoperability

As API gateways evolve to become an integral part of the overall cloud operating system, there is also a push for standardization and interoperability to facilitate this integration. These gateways must no longer operate in isolation. Instead, API gateways should function across various environments, just as cloud-native applications do.

One noticeable shift toward standardization is the Kubernetes Gateway API. As we explore this topic in later sections, we will learn that it tries to standardize API gateway configurations across different API gateway solutions.

From the commoditization and standardization efforts, we can observe an evolution in API gateways toward Cloud Native API Gateways. This section aims to provide an introductory overview of Cloud Native API Gateways. However, it is crucial to note that the depth of this domain extends beyond the scope of this introductory section. In subsequent sections, this thesis will adopt a more granular approach to the different concepts already mentioned, showcasing the roles these concepts play within the overall Cloud Native architecture.

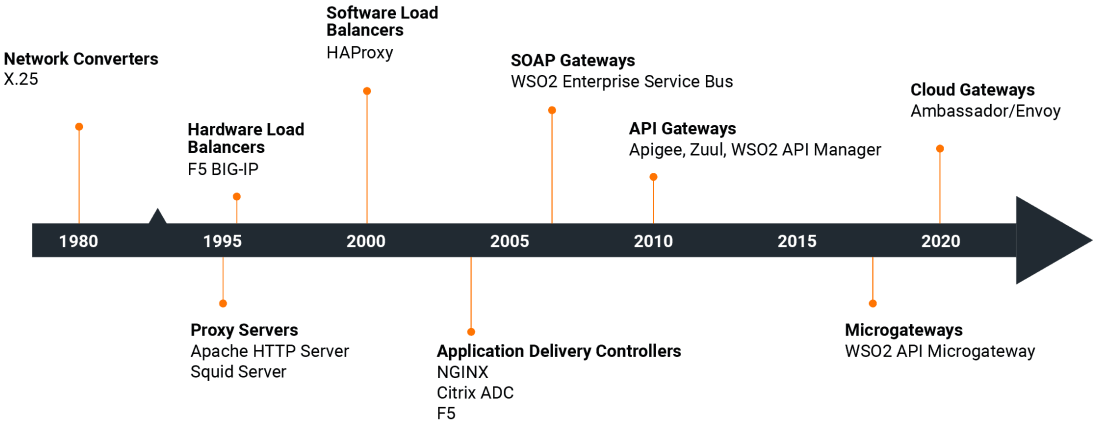


Figure 5: The evolution API gateway solutions (Gunatilaka, 2023)

3 Literature Review

3.1 Evolution of Gateway Architecture

The evolution of gateway architectures can be comprehensively understood within the context of technological advancements and the resulting shifts in application architecture and development workflows. Historically, the design and implementation of north-south traffic solutions have been linked to the application architecture and the technologies in use. As applications evolve, so do the underlying technologies to address the challenges introduced by these new transformations. Therefore, it is important to acknowledge that changes in application architecture and workflows have influenced the API gateway solutions used in cloud-native architecture. This section will focus on this shift in application architecture and its impact on API gateway design.

3.1.1 Monolithic Application Era

During the era where monolithic application architectures dominated, several pivotal developments happened in north/south traffic solutions:

1. **Hardware Load Balancers**: Starting with the hardware load balancers that were the pioneers in the realm of scaling services. Primarily, their role was to ensure that services could handle an increasing volume of requests, thereby ensuring the robustness of applications. Their design was targeted toward system administrators, with an infrastructure-centric focus.

2. **Software Load Balancers**: As the technological landscape evolved, software-based solutions like HAProxy began to emerge. These software load balancers, not only

presented a more cost-effective alternative to their hardware counterparts but also provided enhanced flexibility. However, their primary purpose remained aligned with the earlier hardware solutions: facilitating the scaling of services.

3. **Application Delivery Controllers (ADCs)**: The emergence of ADCs marked a significant evolution in the domain of gateway solutions. Platforms like NGINX paved the way with an enhanced suite of functionalities. These controllers were no longer solely about balancing loads; they now optimized application performance, ensured secure communications between clients and servers, offered SSL and TLS termination, provided security measures, enabled caching, and managed traffic more effectively (Gunatilaka, 2023).

4. **First-Generation API Gateways**: After ADCs, the first API gateway implementations emerged. They extended the basic functionality of ADCs and embedded themselves into API management. This integration offered API gateways with new features such as analytics, developer portals, monetization strategies, and advanced L7 routing. Additionally, these gateways transitioned from an infrastructure focus to being service-oriented. This means that APIs were not just for internal use but were also made available for external users to consume.

The transitions and evolutions in this era can be visualized in (Figure 5), providing an understanding of how gateway solutions have changed in response to changing architectural needs.

3.1.2 Modular Architecture

Following the first generation of API Gateways, the landscape of application architecture began to transform. Applications increasingly adopted a modular design, which eventually led to microservice architecture. This shift also affected the API gateway solutions.

5. API Gateways (2nd Generation): The first evolution of traditional API gateways occurred when organizations realized that all microservices contained similar functionalities. For instance, functions such as authentication, monitoring, routing, and rate limiting were implemented in many services. The API gateways evolved to centralize these features, thus providing a single point for common features in the services. Consequently, these gateways expanded their feature sets, becoming more robust and also heavier in their operations.

Historically, API gateways were centralized, serving as a single point through which all API traffic flowed. However, this centralization posed challenges, especially when applications began to adopt a more modular design inherent to microservice architectures.

The centralized nature of traditional gateways found it challenging to cater to the decentralized and autonomous workflow that cloud-native development and microservices demanded.

6. Micro Gateways: To address the challenges of microservices, microgateways emerged. These microservice gateways were designed to be closer to data sources than those of their traditional counterparts. This, in turn, improved response times due to reduced latencies and streamlined operations.

7. Cloud Native API Gateways: Finally, we come to Cloud-Native API Gateways. These are purpose-built to manage traffic in new cloud-native environments; they leverage container orchestration platforms like Kubernetes and incorporate cloud-native principles in their design to ensure scalability and resilience. As Cloud-Native API Gateways became an integral part of the infrastructure, particularly within Kubernetes, these gateways became inherently closer to the services they communicate with.

API gateway solutions have evolved as applications have transitioned to cloud-native. The gateways, once centralized units, evolved to meet the decentralized, modular structure of cloud-native applications and the demands of full-cycle development by agile teams. In this context, understanding the roles and functions of API gateways within cloud-native applications becomes crucial. In the following sections, we will explore the overall role of API gateways in cloud-native applications and what other solutions we have to meet new demands of cloud native applications.

3.2 API Gateways, Ingress Controllers, and Service Meshes

In cloud-native ecosystems, networking is crucial for orchestrating communication between various services. Previous discussions focused on API gateways, which represent one method of managing communication within microservice architecture. However, they are not the only means of handling communication in cloud-native settings. This section examines critical technologies that facilitate this communication: API gateways, service meshes, and ingress controllers. Each technology serves a distinct purpose, addressing different aspects of network traffic management and service interaction within Kubernetes clusters.

We will examine each solution in turn, providing the necessary background to enable a comparative analysis later in the thesis. The insights gained from this examination of Kubernetes networking technologies will be vital for the discussions in subsequent sections. Understanding the main points and functionalities of each technology equips us to discuss their integration and practical applications in real-world cloud-native environments.

3.2.1 API Gateways in Cloud Native Context

API Gateways, for a significant period, served as the cornerstone for managing north/south traffic in data centres (Palladino, 2023). Their critical role in directing and managing traffic has been well-documented across multiple phases of technological evolution. As we navigated in the historical overview of this thesis, API Gateways have witnessed considerable change in their implementation and design.

The traditional enterprise API gateways, characterized by their centralized nature, were focused on administration. These systems often require the involvement of an API management system to function optimally. Such gateways typically received updates via an admin API, thus providing centralized control over gateway modifications.

Yet, as with any evolving technology, certain inherent challenges have emerged over time with this model (Posta, 2020):

1. **Scalability Issues**: Often perceived as static entities, these gateways frequently struggled to scale efficiently in dynamic cloud environments.
2. **Centralized Deployment**: A highly centralized approach often results in reduced flexibility and adaptability.
3. **Traffic Bottlenecks**: The lack of scalability inherently produced bottlenecks, hindering smooth traffic flow.
4. **Lack of Delegation**: Viewing the entire gateway solution as a monolithic entity often meant its management was assigned to a specific team, creating silos within organizations.
5. **Reliance on Outdated Technologies**: Legacy technologies, though tried and tested, sometimes obstructed innovation and adaptability.
6. **Challenging Automation**: Integrating modern automation techniques proved difficult, given the rigidity of such systems.

An observation from Posta (2023) encapsulates the challenges and evolving needs of API management: "Any API management system must fit within a modern development environment that is often multi-language, multi-platform, and multi-cloud." This statement underscores the importance of API gateways to be more versatile, flexible, and in line with modern development practices.

One feature of Cloud Native API Gateways is their developer-centric approach to gateway management. Unlike the centralized approach of past gateways, these are integrated as part of standard development processes. This integration represents a shift from

viewing API Gateways as isolated, monolithic entities to understand them as flexible components within a broader cloud-native infrastructure. Consequently, they are not always exclusively tied to API management systems.

The transformation of API gateways highlights a pivotal trend: their evolution towards becoming interchangeable components, just like any other element in the system. This commoditization aligns well with the cloud-native approach, which aims for a technology-agnostic way to build applications.

Historically, API gateways have been tailored to address a multifaceted set of challenges and opportunities. At the forefront of these is the concept of "API-as-Product." Here, the API isn't merely a technical interface but a marketable entity. Under this context, the gateway facilitates application governance, enables developer onboarding processes, and even allows for API monetization. The idea is to make APIs available to external users and enrich their interaction with the API management.

API gateways also handle the "Service Connectivity" aspect of applications, meaning that an API gateway acts as a channel that helps connect various services within the application. API gateways offer a suite of features to enhance service interactions, including observability, security protocols, policy enforcement, and rate-throttling.

Lastly, there's an inherent aspect of "API Management Facilitation" to consider. Traditional API gateways often do not exist in isolation. They frequently form a part of a larger API management ecosystem, enabling developers to effortlessly integrate with developer portals and other related tools. This integration signifies that the gateway is part of a bigger system called API management.

3.2.2 Navigating North/South Traffic: Ingress for Kubernetes

API gateways are not just the only option for handling north/south traffic in Kubernetes clusters. A foundational mechanism for this purpose is the Kubernetes "Ingress". As defined by Kubernetes (2023c), Ingress is "an API object that manages external access to the services in a cluster, typically HTTP." Its main role involves facilitating the external exposure of internal services that reside within the Kubernetes ecosystem.

The utility of Ingress is underscored by the capabilities it brings to the table: executing load balancing, ensuring the termination of SSL/TLS, facilitating name-based virtual hosting, and bestowing services with externally accessible URLs. Figure 6 offers a visual representation of Ingress's positioning within a Kubernetes cluster, providing a clearer understanding of its role.

Within the Kubernetes ecosystem, Ingress serves as an elementary tool for managing north/south traffic, its scope can sometimes be restrictive for organizations with more

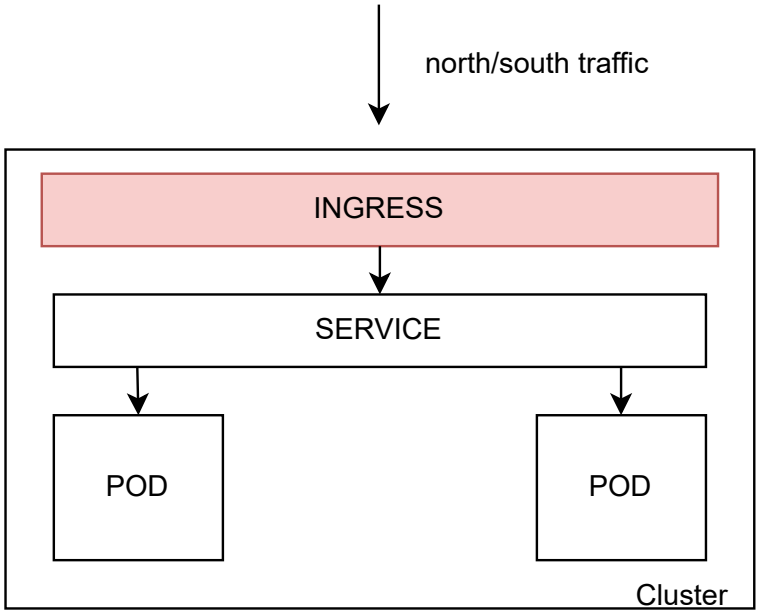


Figure 6: Ingress for Kubernetes

advanced requirements. The inherent limitations in its feature set can pose challenges, especially when a more complicated traffic control, such as %-based canary releases, becomes a requisite.

Kubernetes Ingress itself is just an API that contains a collection of routing rules defining how traffic should be handled in the Kubernetes cluster. Ingress Controllers do the actual work by reading the Ingress information and managing the routing based on these rules. A noteworthy technology used for creating Ingress controllers is Envoy-proxy, which is discussed in detail later in this section.

While the landscape of ingress in Kubernetes has traditionally been diverse, it has also been evolving to meet the growing complexities of cloud-native applications. This evolution is evident in the emergence of the Kubernetes Gateway API, which can be seen as the next-generation Ingress API for Kubernetes and will be the focus of our next discussion.

3.2.3 Kubernetes Gateway API: The Next Generation Ingress API

The Kubernetes Ingress API is not the only mechanism for handling north-south traffic in the Kubernetes environment. A standout API called the Kubernetes Gateway API has emerged as the next generation of this technology. The Gateway API signifies a strategic shift in handling north-south traffic; it doesn't replace its predecessor, however, it enhances ingress with a broader set of features (Kubernetes Gateway API, 2023e).

The Gateway API is an open-source project managed by the SIG-NETWORK community.

It distinctly defines itself as a collection of resources designed to model service networking within Kubernetes, catering to both north/south and east/west (service-to-service) traffic. What makes the Gateway API particularly unique is its role-oriented design. As cited, "Gateway is composed of API resources which model organizational roles that use and configure Kubernetes service networking" (Kubernetes Gateway API, 2023d). This design philosophy emphasizes not just the functionality but also the configurability and manageability of service networking, ensuring alignment with the specific roles and responsibilities within an organization.

Building upon the foundational capabilities of Kubernetes Ingress, the Gateway API significantly expands the realm of functionalities to address a broader spectrum of networking challenges. At its core, the Gateway API maintains the basic features that are familiar to Kubernetes Ingress users. However, it supplements this foundation with advanced routing mechanisms that cater to the complicated needs of modern applications.

A noteworthy enhancement is the Gateway API's refined routing rules. Beyond just domain-based routing, the Gateway API introduces capabilities for HTTP matching, allowing for sophisticated routing based on query parameters or headers. This granular level of routing offers improved traffic management and client request handling. Additionally, the Gateway API natively support a range of different protocols such as gRPC (Kubernetes Gateway API, 2023c).

Yet, what truly sets the Kubernetes Gateway API apart from Ingress API is its emphasis on standardization and extensibility. Although users will need specific implementations to harness the Gateway API's power, its design principles ensure easy extensibility (Kubernetes Gateway API, 2023d). This means that as the networking landscape evolves and newer requirements emerge, implementations can accommodate and integrate additional features with ease.

In essence, the Gateway API is not just about facilitating connections; it's about versatility and adaptability. Kubernetes Gateway API (2023e) showcases its versatility by highlighting varied use cases, from basic north/south traffic management to more experimental scenarios like facilitating communication between multiple applications behind a single gateway or even potential integrations in a service mesh architecture. Each of these use cases underscores the Gateway API's commitment to providing comprehensive networking solutions for Kubernetes, regardless of the complexity of the application landscape.

Gateway API resources

In the Kubernetes Gateway API, the orchestration of networking is abstracted and represented using a distinct set of resources, each serving a specific purpose in the

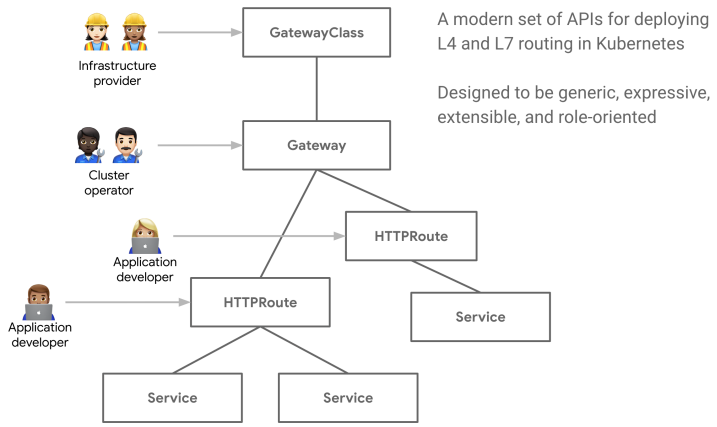


Figure 7: Gateway API uses Kubernetes resources to model API networking (Kubernetes Gateway API, 2023d)

networking lifecycle. The holistic model, as illustrated in Figure 7, provides a comprehensive visual representation of how these resources interplay to deliver the desired networking configurations.

At the foundation of this model is the **GatewayClass**. As articulated by Kubernetes Gateway API (2023d), the GatewayClass serves as a blueprint, defining a homogenous set of gateways that share both configuration and behavior. It contains a shared configuration for different gateways, thus helping to streamline the deployment of multiple gateways with similar configurations.

Building on this foundation is the **Gateway**, which essentially represents an entry point. It handles the crucial task of channelling traffic towards appropriate Services within the cluster, thereby ensuring that client requests are directed to the correct internal endpoints.

Lastly, the **Routes** resource plays a pivotal role in traffic direction. It contains the pathing logic, specifying how incoming traffic, once it reaches the Gateway, should be directed to the various Services. This ensures not only efficient traffic management but also that requests are processed by the appropriate Services, optimizing the end-user experience.

Collectively, these resources present a modular framework for modelling service networking in Kubernetes via the Gateway API, offering a role-based model for designing complex networking paradigms.

Difference between API Gateway and Gateway API

The terms ”API Gateway” and ”Gateway API” are notably similar in their naming, leading to frequent confusion. However, they are two different concepts that should not be mixed up.

An API Gateway acts as an intermediary mechanism, essentially serving as a bridge between backend services and their potential consumers. Its role extends beyond mere traffic forwarding; it gives the networking layer more capabilities like load balancing, request and response transformations, and occasionally, more advanced features such as authentication, authorization, rate limiting, and circuit breaking. This turns the API Gateway into a powerful tool that enhances the capabilities of a backend service.

On the other hand, the Gateway API, specific to the Kubernetes ecosystem, is a set of structured resources crafted to model service networking within Kubernetes. It provides a framework through which Kubernetes networking can be represented and managed, with the Gateway being a principal resource within this framework. The Gateway not only declares the type or class of gateway to instantiate but also outlines its configuration. When providers implement the Gateway API, they are essentially architecting Kubernetes service networking in a manner that is both expressive and role-centric.

In summation, while the API Gateway is a conceptual tool encompassing a wide array of traffic management features, the Gateway API is a Kubernetes-specific interface, designed to model service networking. Yet, it's relevant to understand that implementations of the Gateway API can indeed function as API Gateways, bridging the conceptual gap between these two terms.

Implementations

The Kubernetes Gateway API itself is just a set of Kubernetes resources. For functionality, it needs an implementation for it. As of the time writing this, there are currently over 20 distinct implementations of the Gateway API, each at different stages of development, ranging from preview to beta to alpha (Kubernetes Gateway API, 2023b).

It's important to realise that the Kubernetes Gateway API acts as the backbone for Kubernetes networking solutions. Whether it's service mesh providers or API gateway vendors, the Gateway API serves as the foundational structure upon which these solutions are built. It provides implementors with a standardized and extensible API to forge Cloud Native API gateways or other related traffic management solutions for applications.

Discussing Ingress and the Kubernetes Gateway API in this thesis is motivated by their pivotal roles in the evolving landscape of cloud-native technologies. The emergence of the Kubernetes Gateway API underscores the trend where API gateways are progressively becoming an integral, almost commoditized, component of the infrastructure. This standardization increases consistency between implementations. Both the Ingress API and Gateway API showcase that north/south traffic can also be managed from inside the clusters. The extensibility of the Kubernetes Gateway API also ensures that further innovations in traffic management solutions can happen in the cloud-native domain.

3.2.4 Service Meshes

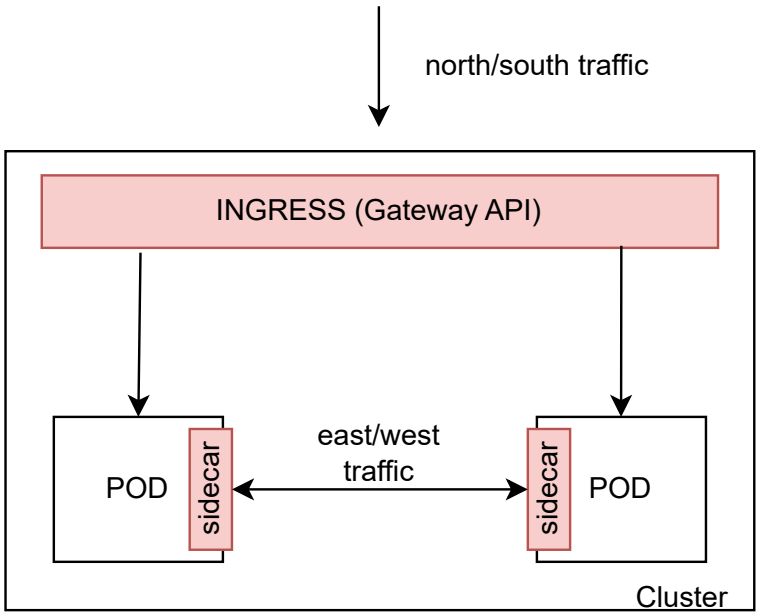


Figure 8: Sidecar implementation for service mesh

Service meshes, while previously mentioned in various sections of this thesis, are now being brought to the forefront for discussion. Service meshes have become an essential component within the cloud-native landscape, especially when the focus is on service-to-service communication. At their core, service mesh implementations offer routing, observation, and secure transmission of traffic between services (James Gough, 2020). Unlike API gateways, which focus on the management of north/south traffic, service meshes are designed to cater to east/west traffic in cloud-native architecture.

The central goal of service meshes is to decouple and externalize common service-to-service communication patterns. Instead of embedding functionalities like authentication and authorization within each microservice, a service mesh employs a proxy mechanism to handle these common features. This proxy effectively manages repetitive tasks while ensuring consistency and efficiency. A notable advantage of service meshes is their technology-agnostic nature: regardless of whether microservices are developed in Java, C#, or any other programming language, the service mesh proxy can seamlessly integrate with each service. This means that services do not need to independently implement common functionalities such as monitoring when using service mesh.

Service meshes centralize the management of internal communication between services, as depicted in Figure 8. Given that the data plane proxy operates in tandem with every service instance, service meshes can be characterized as a decentralized deployment, optimizing scalability and robustness (Palladino, 2023).

It is also crucial to recognize that service meshes, which were traditionally dedicated to managing east/west traffic, are increasingly crossing boundaries and integrating with north/south traffic solutions. This shift is driven by a recognition within the industry that managing two disparate systems for internal and external traffic may not be the most efficient approach. Teams are seeking to streamline their technology stacks, looking towards solutions that can handle both traffic flows. As such, service mesh implementations are progressively expanding their feature sets to better accommodate north/south traffic, aiming to offer more capabilities.

An example of the convergence of north/south and east/west traffic is the adoption of the Kubernetes Gateway API in service mesh implementations. This integration with the Gateway API has two aspects: standardizing the service mesh implementations through the "Gateway API for Service Mesh" (GAMMA) initiative (Kubernetes Gateway API, 2023a), and unifying the configuration of service meshes with API gateways. Consequently, service mesh providers have also begun creating Gateway API implementations tailored to their specific use cases.

This convergence is significant as it begins to blur the previously clear distinction between API gateways and service meshes. The emerging solutions are starting to view networking as a holistic challenge within the Kubernetes ecosystem. Instead of separate strategies for east/west and north/south traffic, the emerging solutions focus on "Container Networking" as the guideline. This approach does not differentiate between the two directions of traffic. Instead, it views them as parts of the larger picture of "Kubernetes networking".

Service mesh features

As we shift focus to the functionalities commonly found in service meshes, it's crucial to acknowledge that while features may vary across different implementations, there are core capabilities that are widely expected. These features not only define the scope of the service meshes but also highlight the areas of overlap with API gateways, offering insights into the converging landscape of cloud-native networking solutions.

Service meshes typically offer a suite of service connectivity features that facilitate secure and efficient microservice communication. Key aspects of service connectivity include:

- **Observability**: Providing insights into the performance and health of services, often through detailed metrics, logs, and traces.
- **Rate Limiting**: Regulating the flow of traffic to services, ensuring stability and mitigating the risk of system overloads.

- **Policies**: Enabling the enforcement of governance and operational rules at the communication level between services.
- **Security**: Ensuring secure service-to-service communication with mechanisms like mutual TLS, often managed transparently by the service mesh.

Additionally, service meshes address Layer 4 (L4) traffic features, managing the transmission of data packets based on the network layer of the OSI model, which deals with protocols like TCP and UDP. This low-level traffic management allows service meshes to handle not just HTTP(S) but also a broader range of network protocols, making them a versatile choice for multi-protocol environments.

Expanding on service connectivity, service meshes typically contain:

- **Service Discovery**: Automatically detecting services within the network, facilitating dynamic, scalable architectures.
- **Load Balancing**: Efficiently distributing incoming traffic across multiple service instances to optimize resource use and response times.
- **Traffic Control/Shaping**: Directing and manipulating traffic flow to meet reliability and performance objectives.
- **Traffic Metric Collection**: Gathering data on traffic patterns and usage, essential for monitoring and planning.
- **Service Resilience**: Providing mechanisms like retries, circuit breakers, and timeouts to improve the robustness of service interactions.

The provision of these features indicates the integral role service meshes play in the modern cloud-native ecosystem. While the primary focus of service meshes is on east/west traffic, they can also implement their own solutions for north/south traffic, thereby blurring the lines between internal and external service management.

3.2.5 Service Mesh Implementations

Now, we will shift our focus from an overview of service mesh to individual implementations, to further understand the implications of service mesh for Kubernetes networking. This section demonstrates that there is a noticeable difference between various service mesh solutions, which is inherent in the underlying architecture and technology choices of each service mesh. Understanding these differences provides readers with the knowledge to choose among various service meshes, by comprehending the benefits and challenges of each implementation.

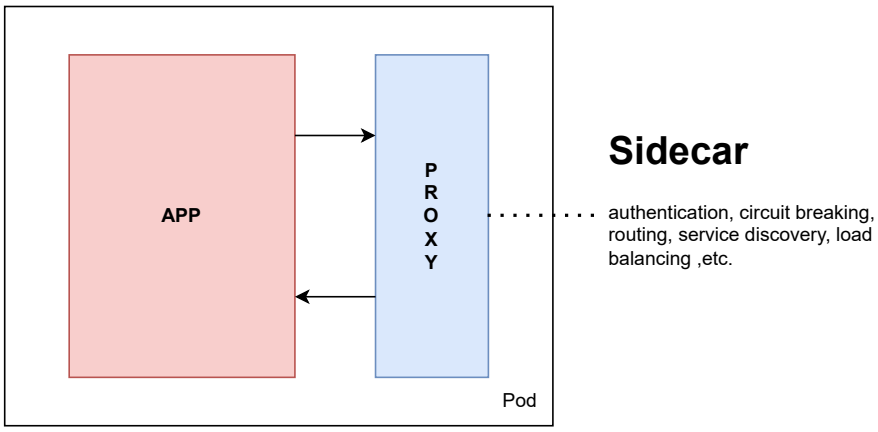


Figure 9: Sidecar is the process next to service in a pod

The sidecar pattern

First, I will introduce the most popular service mesh implementation pattern called ”the sidecar” pattern, which is used in many popular service mesh implementations such as Istio, Linkerd, and Consul (Duarte Maia and Figueiredo Correia, 2022). The sidecar pattern is an approach to service proxying where the proxy is situated right next to the service (Davis, 2019). Sidecars themselves are separate processes running alongside the main process, which manages the actual application logic. The idea of the sidecar model is to create a proxy that centralizes common functionalities of services, such as authentication, routing, or monitoring. This proxy can then be integrated into any service, regardless of the programming language used by the service. As a result, the sidecar pattern augments the application at runtime without requiring changes to the actual application code. However, the sidecar remains within the same operational and security context as the application itself (Morgan, 2023).

In practice, both the service and the sidecar reside within the same Kubernetes pod. Consequently, every pod in a Kubernetes cluster contains a service mesh network proxy, which manages the traffic directed to the services. Figure 9 visualizes this setup within a Kubernetes pod.

The service mesh is designed for service-to-service communication; however, applications also need to be accessible from outside the Kubernetes cluster. I have depicted the common traffic flow of service-to-service communication in Figure 10. A central concept to understand in service mesh implementation is the distinction between the data plane and the control plane. The data plane is responsible for managing traffic. Traffic first arrives at the data plane from the ingress, and then it reaches various service proxies which handle the traffic directed to the services. Additionally, there is the control plane, whose primary job is to configure the service mesh proxies and thereby provide information about the system.

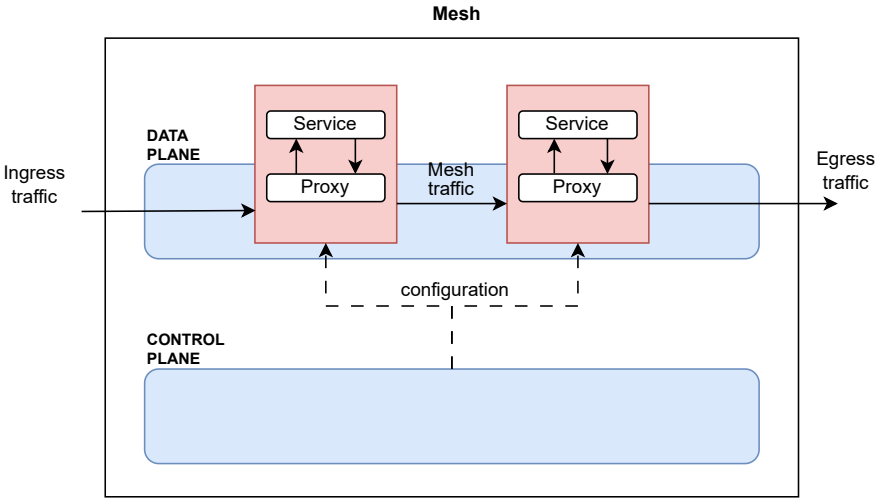
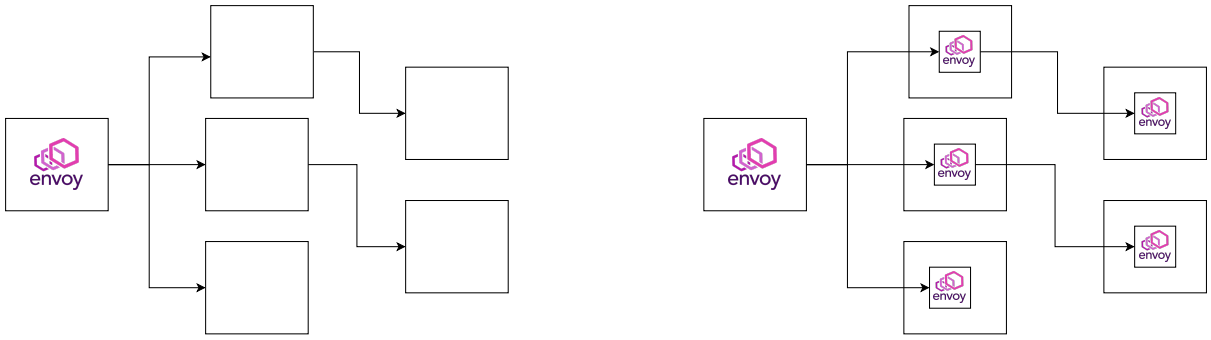


Figure 10: Service mesh proxies the traffic in data plane to services

There are multiple benefits to using service meshes. First, a sidecar implementation does not involve a network hop in communication, unlike API gateways. For example, with API gateways, requests must always pass through the gateway to reach the service. These gateways are centralized components that handle traffic management, meaning there is always a network hop when using an API gateway. In contrast, a sidecar service mesh does not require an extra network hop because the sidecar resides at the same IP address as the service. As a result, implementing common features in the sidecar can increase application performance due to more streamlined communication.

Another benefit of using a service mesh to handle common application functionalities is the centralization of these functionalities, as previously mentioned. The sidecars themselves act as proxies inside the pod, allowing the sidecar implementation to be written in a different programming language than the service. This means that only one sidecar implementation needs to be created, which can be universally applied within the Kubernetes network, regardless of the technologies used in the services.

However, historically, sidecar implementations have also had problems and challenges (Morgan, 2023). These issues are usually linked to the underlying infrastructure and how sidecars function within the pod. First, upgrading a sidecar necessitates restarting the entire pod in question. This can be problematic because the actual logic inside the service may not require any changes, but it is also restarted when the sidecar needs an upgrade. Second, sidecars can prevent Kubernetes Jobs from terminating. Kubernetes Jobs are tasks that run to completion (Kubernetes, 2023b). Using a sidecar can cause these Jobs to never terminate because the sidecar itself does not know whether it should terminate. Lastly, sidecars can lead to race conditions within the Kubernetes cluster. If containers in a Kubernetes pod start in the wrong order, such as the service starting faster than the service mesh proxy, it can result in issues. In these situations, the service is ready



(a) Envoy as edge proxy (API gateway)

(b) Envoy as sidecar service mesh

Figure 11: Envoy has diverse use cases

to receive requests, but the proxy is not yet operational, causing communication to the service to fail.

There has been discussion in the Kubernetes community about fixing the sidecar problems, and as of Kubernetes version 1.2.8, there is finally native support for sidecars in Kubernetes (Neal et al., 2023). This means that sidecars are becoming more of a feature than a pattern in Kubernetes, thus improving the service mesh implementation using the sidecar pattern. This native support also addresses some of the problems associated with older sidecar container implementations, such as race conditions during container startup (Morgan, 2023).

The next section will delve more deeply into one of the technologies used to create both service mesh and API gateway implementations. This will help us understand the differences between these approaches and provide a more in-depth understanding of service meshes.

Envoy proxy

Envoy proxy has been one of the most influential technologies in the domain of Cloud Native API Gateways and service mesh implementations due to their wide use in popular solutions. Envoy is an L7 proxy and communication bus designed for service-oriented architectures, such as Kubernetes (Proxy, 2023b). Envoy proxy is also a graduated CNCF project (Cloud Native Computing Foundation, 2023b). Its influence stems from its use as the underlying technology in many service mesh and API gateway solutions, including Istio, Ambassador, Consul, and Kuma (Proxy, 2023a). Given that Envoy proxy is capable of implementing both service meshes and API gateways, it is safe to say that it offers a comprehensive feature set.

Envoy proxy's use cases are very diverse. First of all, many API gateway solutions are built on top of the Envoy proxy. In this use case, the Envoy proxy works as a proxy

at the edge of the cluster, directing traffic to the services. This is depicted in Figure 11a. Example of Envoy proxy-based API gateways is the Edge Stack API Gateway (Ambassador Labs, 2023a).

Envoy's diverse feature set also makes it a great choice for the underlying technology in service mesh implementations. In the service mesh architecture, the Envoy proxy functions as a sidecar implementation, thus enabling service-to-service communication inside the cluster (see Figure 11b). An example of an Envoy proxy-based service mesh implementation is Istio (Istio, 2023).

Being able to create both the service mesh and the API gateway solution using the same underlying technology has its own implications. First of all, this will make integrations between service mesh solutions and API gateways easier. What this means is that communication between the service mesh and the API gateway can be easily established if both use Envoy proxies. Using the same technology for both the service mesh and the API gateway is also beneficial in reducing the overhead in communication (Farnham, 2023).

Another implication is that it becomes easier to start implementing a Kubernetes networking solution that can handle both north/south and east/west traffic. This has been particularly evident in service mesh solutions, which have begun to experiment with providing basic north/south traffic management within the service mesh solution. The Envoy proxy is a great choice if both solutions are needed because the provider can use the same underlying technology for both north/south and east/west solutions.

In summary, the Envoy proxy is a multi-purpose proxy technology that is used in many popular service mesh and API gateway solutions. It functions as the underlying technology to handle traffic to services. Its versatility showcases that the boundary between service mesh and API gateway solutions is becoming increasingly blurry due to the use of the same underlying technology in both solutions while offering a similar feature set.

Envoy Gateway

There's another technology created by Envoy that is also worth talking about due to its unique position in Cloud Native API Gateway solutions: Envoy Gateway. Envoy Gateway is an open-source API gateway product powered by Envoy proxy (Envoy, 2023a). The Envoy proxy itself is quite a complex technology due to its flexibility. Envoy Gateway attempts to simplify the use of Envoy proxy by creating a new API that manages the Envoy proxies. Consequently, this lowers the barrier to using Envoy proxies.

Since Envoy Gateway is based on Envoy proxies, it falls under the CNCF's Envoy proxy

umbrella project. This also signifies that it is a Kubernetes-supported technology for API gateways. Kubernetes plays a crucial role in Envoy Gateway, as it implements the previously mentioned Kubernetes Gateway API (Envoy, 2023a). Envoy Gateway utilizes Gateway API resources to provision and configure the managed Envoy proxies within the application (Envoy, 2023b).

Envoy Gateway includes many features that leverage Envoy proxies. Firstly, Envoy Gateway supports both L4 and L7 OSI layer routing. It also offers basic functionalities common to all API gateways, such as routing, observability, and security.

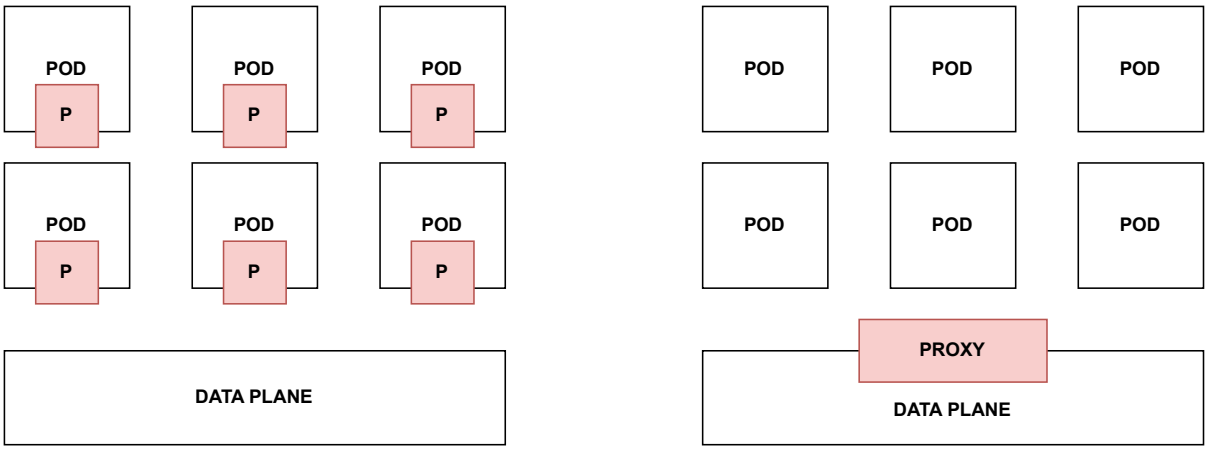
What truly distinguishes Envoy Gateway from other solutions is its extensibility. The previous section mentioned that there are already API gateways and service meshes based on Envoy proxies. Envoy Gateway does not aim to cannibalize the vendor market; instead, it seeks to establish a more robust API upon which other API gateway vendors can build. This approach means that instead of developing an API gateway solution directly on top of the Envoy proxy, an API gateway vendor can utilize Envoy Gateway to manage the Envoy proxies within their API gateway solution.

The extensibility of Envoy Gateway has showcased an important trend in the evolution of API gateways, which is the standardization and unification of API gateway solutions. Envoy Gateway aims to eliminate the need for different API gateway vendors which use Envoy proxies to duplicate basic features (Klein and Reisz, 2022). This means that Envoy Gateway provides a core API that can already be used by these vendors for basic functionalities. Consequently, vendors using Envoy proxy can focus on adding unique value to their API gateways, rather than replicating functionalities common across the industry. Therefore, Envoy Gateway is an API gateway that contains only the essential functionalities of such solutions. The extensibility of Envoy Gateway enables other vendors to build their own customized API gateway solutions on top of it. This, in turn, will allow vendors to work in a higher-management plane layer for Envoy proxies (Envoy, 2023a).

Linkerd

In the current state of service mesh solutions, there are three main challenges that service meshes face: support for high performance, adaptability, and availability (Li et al., 2019). So far, our focus in the service mesh realm has been on Envoy proxy-powered solutions, which aim to address the adaptability challenge by offering an extensible, multi-purpose proxy solution. However, extensibility and a wide range of use cases also affect performance, because the technology itself is not focused on sidecar service meshes.

However, even though the Envoy proxy is a great technology for creating service meshes, it is not the only option. Linkerd is another service mesh solution, which focuses on



(a) The sidecar pattern contains the sidecar proxy at every node

(b) The sidecarless pattern contains single proxy per node

Figure 12: Difference between sidecar and sidecarless pattern inside Kubernetes node

performance and simplicity in management (Linkerd, 2023). It is also a CNCF-graduated service mesh product, similar to the Envoy-proxy-based Istio (Cloud Native Computing Foundation, 2023b).

Instead of using Envoy proxies, Linkerd employs its own proxy, written in Rust, known as Linkerd2-proxy (Linkerd, 2023b). The primary aim of this choice is to provide a simpler and more lightweight proxy solution than the Envoy proxy for sidecar deployments. Since Linkerd's implementation is more lightweight with a focus on performance, its feature set reflects this choice. Linkerd prioritizes performance and simplicity, resulting in a more limited feature set compared to solutions using Envoy proxies. In contrast, the Envoy proxy emphasizes flexibility and serves as a general-purpose solution for proxy technology. Consequently, it showcases higher latency in request handling compared to Linkerd (Adinegoro et al., 2022).

As of writing this section, the focus of Linkerd has been solely on service-to-service communication and it has not yet provided an ingress controller. However, it is designed to integrate with existing ingress controllers. Moreover, the Linkerd roadmap also includes mentions of ingress and egress traffic controls (Linkerd, 2023a). This expansion indicates that service mesh providers are gradually extending their use cases beyond just managing east-west traffic.

The sidecarless patterns

While the sidecar pattern has been the most popular approach in the service mesh realm, it is not the only method for handling service-to-service communication within service meshes (Zhu et al., 2023). The remainder of this section introduces the concept

of sidecarless service meshes and explores how they differ from sidecar-based solutions.

There are notable challenges inherent to the sidecar pattern. These challenges arise because every pod in a sidecar pattern service mesh must have its own sidecar to facilitate communication. As such, each sidecar in the node requires system resources to operate. As the number of pods increases, so does the resource demand for the service mesh.

A pattern that has become more popular for service meshes without sidecars is the "node agent" pattern (Duarte Maia and Figueiredo Correia, 2022). In this approach, instead of assigning a proxy to every pod on a Kubernetes node, a single proxy is created for the node. This node then acts as a proxy for all services running on it, as depicted in Figure 12b. While the specifics of how a node agent is created and functions may vary between implementations, the concept of a per-node proxy remains consistent.

The node agent pattern offers multiple benefits over the sidecar pattern (Duarte Maia and Figueiredo Correia, 2022). The most noticeable difference is in resource utilization. In the node agent pattern, resources are needed for only a single proxy that manages communication with all the pods on the node. Because the node agent pattern requires only one proxy, it is easier for organizations to implement, as it needs less configuration (Duarte Maia and Figueiredo Correia, 2022). Additionally, it is simpler to manage because all the logic is centralized in a single entity.

Next, we will explore some of the sidecarless implementations of service meshes. This will provide a good overview of the overall service mesh landscape, which is crucial for beginning to compare different approaches to Kubernetes networking. I will introduce two significant service meshes with sidecarless implementations: Istio Ambient Mesh and Cilium Service Mesh.

Istio Ambient Mesh

Drawing from the previous discussion on sidecarless service meshes, we will first introduce the Istio Ambient Mesh, a sidecarless Istio service mesh (Istio, 2023). Previously, Istio had only offered a sidecar implementation of the service mesh, but the expanding needs of users for a more resource-efficient and easy-to-use solution have motivated Istio to explore sidecarless options for service meshes (Howard et al., 2023).

The adoption of the sidecar pattern for Istio has not been without its problems and challenges. The creators of Istio have identified three notable issues in the sidecar model: race conditions, misinterpretation of L7 protocols, and the "all-or-nothing" proposition (Sun and Posta, 2022).

1. **Race-Condition Issue**: Refers to the already mentioned scenario where the workload becomes available before the sidecar proxy in the pod.

2. **Interpretation of ISO Layer 7 Protocols**: Problems may arise from the sidecar's interpretation of these protocols, especially when libraries incorrectly implement the protocol but still utilize the sidecar for communications.

3. **"All-or-Nothing" Proposition of Sidecar Proxy**: In the sidecar model, there's no way to adopt only certain features of Istio. For instance, even if a user needs only the mTLS feature, the L7 features are also implemented in the proxy, regardless of their use (Howard et al., 2023). This implies that you either implement all the features in the proxy, or you do not use the sideproxy at all.

The Istio Ambient service mesh addresses these problems by creating a sidecarless proxy architecture, which separates the L4 and L7 capabilities of Istio (Sun and Posta, 2022). This new Istio data plane, not using sidecars, eliminates the race condition issues associated with pod starting workload before sidecar proxy. By separating L4 and L7 capabilities, Istio can utilize L4 features in the mesh without necessarily implementing L7 features. This approach is beneficial for circumventing misinterpreted L7 protocol issues, as it allows for opting out of L7 features if they are not needed. Additionally, the separation of layers addresses the "All-or-Nothing" problem by enabling incremental adoption of L7 features. For instance, we can initially use only the Layer 4 mTLS capability in the ambient mesh and later, if needed, adopt more advanced L7 features.

However, the design decision for a sidecarless service mesh has been to ease the adoption, operation, and maintenance of the service mesh (Sun and Posta, 2022). Sidecarless architecture also provides increased resource efficiency, better performance for L4-only capabilities, and separation of application code from the data plane (Sun and Posta, 2022).

Cilium Service Mesh

As we have already explored different approaches to service meshes, it becomes clear that each offers unique capabilities and methods for managing network communication in cloud-native environments. While previous sections have introduced various service meshes and their functionalities, we now turn our focus to a particularly distinctive solution in this field: Cilium. What sets Cilium apart from the other solutions already mentioned is its use of eBPF technology, granting it a unique position in the realm of service meshes.

Earning its status as a graduated project within the CNCF (Cloud Native Computing Foundation, 2023b), Cilium's uniqueness lies in its core reliance on the Extended Berkeley Packet Filter (eBPF). This technology sets it apart from traditional service meshes, which mainly rely on standard networking and security mechanisms. By leveraging

eBPF, Cilium introduces a new approach to managing and securing service-to-service communication within Kubernetes clusters.

To understand Cilium, it's essential to understand the underlying mechanics of eBPF. Conceptualized as a lightweight execution environment inside the Linux kernel (Sedghpour and Townend, 2022), eBPF allows for the execution of sandboxed programs directly within the kernel space (Vieira et al., 2020). This capability has many implications: it provides a means for applications to perform networking and security operations at the kernel level, thereby bypassing the performance constraints of user-space solutions.

The integration of eBPF technology enables Cilium to offer enhanced network performance, dynamic policy enforcement, and more efficient load balancing due to working directly in the kernel space. These features are not only important for optimizing network traffic but also for ensuring robust security within cloud-native architectures.

Building on our understanding of eBPF and its integration into Cilium, we can clearly see how Cilium sets itself apart from other service mesh implementations. The key difference lies in the depth at which it operates within the overall system.

Cilium brings the service mesh capabilities to the Linux kernel level, a significant shift from the user-space level at which most service meshes operate. This kernel level functionality enables Cilium to interact more directly and efficiently with the underlying system.

To support more advanced features, such as those at Layer 7, integration with proxy technology is necessary. Cilium facilitates this by allowing integration with service mesh providers specifically for handling L7 traffic features. For instance, Cilium employs the Envoy proxy as a fallback solution to manage traffic when more advanced features are required. This approach enables Cilium to leverage the strengths of both eBPF at the kernel-level and proxy technology at the user-space level, creating a more comprehensive solution for Kubernetes networking.

Building on the discussion of Cilium, it's noteworthy that its use also enables the benefits of the sidecarless proxy model, previously outlined in the thesis. This approach eliminates race conditions during pod startup by using a single proxy. It also reduces operational complexity, allowing for easier management of network policies. Furthermore, the sidecarless model in Cilium enhances resource efficiency, conserving memory and CPU, which is especially important in large-scale applications. Lastly, this model contributes to improved performance thanks to its direct kernel-level integration through eBPF.

A significant part of Cilium's functionality is its ability to integrate with other service mesh providers. This integration allows Cilium to manage lower-level L4 traffic within the mesh while delegating more advanced use cases to the service mesh proxy. This setup enables users to also use other service mesh control planes to manage the proxies

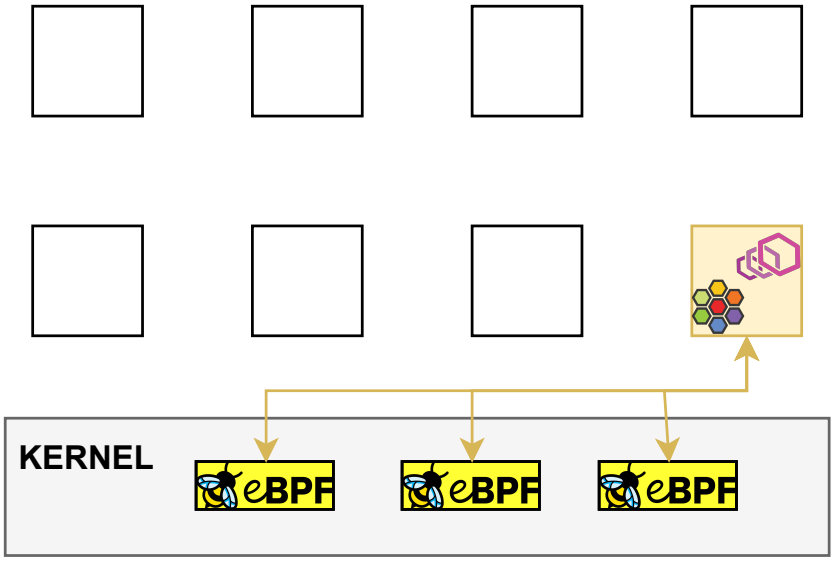


Figure 13: Overview of Cilium architecture inside Node

effectively. Cilium creates a Cilium agent that runs on each node in the cluster. This Cilium agent manages eBPF programs and handles the L7 traffic within the node. This is also depicted in Figure 13.

The use of eBPF in Cilium Service Mesh offers an efficient way to handle traffic, yet it has its limitations. These include eBPF being only a subset of the C language, having limited stack space, and not allowing open-ended loops in the program (Vieira et al., 2020). Due to these constraints, relying solely on eBPF for a service mesh is not a feasible solution. As a fallback, Cilium uses a proxy to handle mechanisms that are not possible in the limited kernel space.

The features that eBPF can efficiently handle include load balancing, mTLS, observability of the services, L3/L4 forwarding, and networking policies (Graf, 2022). However, for advanced features like L7 load balancing, ingress control, rate-limiting, and TLS termination, a proxy is necessary.

4 A Comparative Analysis

API gateways have evolved, adapting to meet the needs of modern application development. Similarly, ingress and service mesh solutions have advanced to address the specific challenges presented by traffic in cloud-native environments. In the last section, we introduced all of these concepts, and it has already become noticeable that, even if the intended use cases might differ, the capabilities of these solutions overlap with each other. This section highlights this overlap of capabilities. The goals of this section are as follows:

1. Highlight the overlap of capabilities more evidently.
2. Showcase the impact of standardization in Kubernetes networking.

As we explore in this section, we see that the distinctions between these technologies have started to blur. They increasingly overlap in terms of capabilities, suggesting a convergence in their functionalities and use cases. An important question arises from this discussion: Which technology is best suited for an organization's traffic management needs?

When we explored the distinctions between different approaches to Kubernetes networking, we noticed that distinguishing between solutions is becoming less clear. If different tools achieve the same goals, regardless of their names, we should focus more on the results than on their definitions. This section aims to underscore this point: the heart of the matter is not in the labels we attach to the tools, but in the roles these technologies play.

We begin this section by examining the differences among similar solutions in the section 4.1, such as how various API gateway solutions differ in the bigger picture, to highlight their unique features and use cases. Following this, we contrast these solutions against each other in section 4.2, providing insights into their roles and interactions within a Kubernetes environment. Finally, in section 4.3, we discuss the impact of standardization on these networking solutions, highlighting its implications for future development. This analysis aims to provide clarity and guidance for navigating the complex landscape of Kubernetes networking.

4.1 Comparison of Similar Solutions

4.1.1 API Gateways

Our comparison of different solutions begins with the focal technology of this thesis: API gateways. We have already explored the variety of features that can be implemented with API gateways and their overall role in a system. However, we have not yet examined the extent to which real API gateway solutions include all the theoretically possible features.

In this section, we aim to explore how decisions made by API gateway vendors influence the range of use cases for which the API gateway can be used. This analysis enables us to compare different approaches to API gateways and understand what benefits an API gateway solution brings to organizations.

Platform API gateways and commoditized API gateways

The first notable aspect to consider when comparing API gateway solutions is the extent to which the API gateway is integrated into the overall API management solution. In the literature review section of this thesis, we examined the evolution of API gateway solutions. We learned that API gateways have transitioned from centralized solutions, which were integral to API management systems, to Cloud Native API Gateways. In centralized API gateway scenarios, the API gateway often forms a core part of the API management system offered by the vendor, rather than being a replaceable component of the infrastructure. This is why, in this section, we will refer to these API management API gateways as "platform API gateways" due to their role as part of the API management platforms.

The platform API gateways, which are centered on API management, lead to use cases that are more aligned with the "API-as-a-Product" concept. These API gateways can be hosted in various ways, such as outside or inside a cluster. However, since the API gateway is an integral part of the API management system, the entire API management setup is installed along with the API gateway. This means that in cases where the hosting is done inside the cluster, the whole API management system is deployed to the cluster, as opposed to only using the features that are needed by the organization.

In the API Management section of our background section, we discussed the various features of API management. Platform API gateways embody features that are crucial to the "API-as-Product" capabilities. These platforms offer a comprehensive set of features that elevate APIs from simple functional components to marketable products. Key features of these platform API gateways include, but are not limited to, developer portals, API documentation sites, API monetization tools, and usage throttling mechanisms.

However, it's important to note that the challenges addressed by these features can also be tackled without fully adopting an API management platform as the API gateway solution. This can be achieved by addressing each issue individually. For instance, developer portals and documentation sites don't necessarily have to be integrated components of the API gateway solution. Instead, they can function as separate entities within the system, each implemented using distinct technologies.

The platform API gateways are not the only option, as we have seen in previous sections of this thesis. API gateways within Kubernetes clusters have become increasingly

commoditized, as exemplified by solutions based on the Envoy proxy. These gateways represent a more modular approach to API gateways, distinct from the solutions integrated with larger platforms like platform API gateways. Examples of these approaches include Cloud Native API Gateways such as the Edge Stack API Gateway (Ambassador Labs, 2023b), EnRoute Gateway (Saara Inc., 2023), and Gloo Gateway (Solo.io, 2023).

A key characteristic of these commoditized API gateways is their seamless integration with Kubernetes environments. This integration underscores their operational flexibility, making them an excellent choice for modular architectures, such as cloud-native architecture.

Unlike platform API gateways, which are often integrated into comprehensive API management solutions, commoditized gateways are notable for their independence. They function as standalone components that can be easily replaced or customized, providing significant flexibility and customization options for the system.

In contrast to platform API gateways, which are developed with the "API-as-Product" concept and include features like monetization and developer portals, commoditized gateways focus on delivering essential functionalities. These essential features include authentication, observability, and routing, without the extra features found in the platform gateways.

To conclude, the key difference between commoditized API gateways and platform API gateways lies in their integration with larger systems, their approach to API management, and their adaptability to different use cases. These differences between the two approaches are shown in Table 1. As the landscape of Cloud Native API gateways continues to evolve, understanding these differences becomes crucial in selecting the most suitable option for an organization's needs.

Underlying technology differences

Another viewpoint for comparing API gateway solutions in cloud-native architectures concerns the underlying technology used to create the API gateway. This means that the proxy technology differs between gateway implementations, and the choice of proxy impacts the API gateway.

In the service mesh section, we already mentioned Envoy Proxy and its impact on service meshes. Envoy Proxy can also be used for gateway solutions, making it a widely used

Table 1: Comparison of Platform and Commoditized API Gateways

	Platform API Gateway	**Commoditized API Gateways**
Integration Approach	Tightly integrated within an API management solution	Flexibly integrated into the overall system architecture
Primary Feature Focus	Comprehensive "API-as-Product" features (e.g., developer portals, monetization)	Focus on centralizing common service features (e.g., routing, load balancing)
Configuration Style	Configurations are typically managed via the API management platform	Configurations are managed using platform-independent files (e.g., Kubernetes Custom Resource Definitions)
API Management Implementation	API management is typically bundled with the API gateway as part of a unified platform	API management features are implemented through separate, standalone components outside the API gateway

Table 2: Comparison of underlying technology choice

	Own proxy	**Existing proxy (e.g. Envoy Proxy)**	**Existing gateay (e.g. Envoy gateway)**
Customizability	High (fully customizable)	Moderate (depends on the proxy)	Low (limited by the extension points)
Complexity	High (requires more development)	Moderate (requires development in proxy level)	Low (basic capabilities provided)
Performance	Variable (depends on implementation)	High (optimized and proven)	High (optimized and proven)
Time to market	Longer (requires more development)	Shorter (quicker integration)	Shortest (ready-to-use solutions)
Integration with Ecosystems	Custom (needs specific integration)	Broad (supports many ecosystems)	Limited (to gateway's capabilities)

proxy technology for implementing API gateway solutions in Kubernetes. By building on top of a widely-used proxy technology, such as Envoy Proxy, an API gateway vendor can utilize the basic capabilities that the underlying technology offers.

The use of an existing proxy as the underlying technology enables API gateway vendors to focus more on value-added features, such as advanced security patterns or the addition of API management capabilities to the solution. For instance, Gloo Gateway utilizes Envoy Proxy as its underlying proxy technology while providing API management features (Solo.io, 2023).

However, even though building on top of existing proxy technology enables gateway vendors to more quickly start providing value-added features, creating their underlying technology can also be beneficial. As discussed in the service mesh section, developing a proprietary solution can enhance performance metrics or allow for more customization in the underlying technology. For example, the KrakenD gateway uses the Lura project to build its gateway solution. This approach has allowed KrakenD to provide a more performance-efficient solution than Envoy-based API gateway solutions such as Tyk and Kong (KrakenD, 2016).

The proxy layer is not the only option for building an API management solution. In the Envoy section, we introduced the Envoy Gateway, which allows other API gateway providers to extend the Envoy Gateway according to their own needs. This means that API Gateways can be built on top of other API Gateways, further assisting gateway vendors in focusing on features that are most valuable for their customers. This approach also implies that basic API gateway functionalities are already handled by the underlying API gateway technology, with extensibility enabling further development. Differences between the approaches are showcased in the Table 2.

4.1.2 Service Meshes

In this section, we continue our comparative analysis of the service mesh landscape. Although service meshes share the common goal of facilitating service-to-service communication in microservice architectures, they significantly differ in their approach and execution compared to API gateways. This analysis aims to thoroughly explore the distinctions between service mesh implementations that were introduced in the Literature Review section. By comparing the architectures, features, and trade-offs of popular service meshes, this discussion seeks to provide a comprehensive understanding of their suitability for various operational environments.

Underlying technology differences

The choice of underlying technology significantly influences the capabilities and overall implementation strategy of a service mesh. While previous sections have showcased various service mesh implementations, a deeper exploration into how underlying technologies influence these implementations has not yet been done. This section aims to bridge this gap by investigating the implications of different technological foundations used in service meshes.

Unlike API gateways, where the choice of technology has significant but often subtle effects, the impact in the context of service meshes is more visible. This distinction arises from the diverse range of technologies that various service mesh implementations use, each bringing unique strengths and limitations.

One option, as with API gateways, is to use existing proxy solutions for service mesh implementations. Using a proven proxy solution offers similar benefits in the context of service meshes, including ease of adoption, proven performance, and extensive support for the technology. This makes established proxy technologies, such as the Envoy proxy, a strong candidate for creating a service mesh solution. As demonstrated by Istio's use of Envoy sidecar proxies, utilizing existing proxies enables the creation of service meshes with a vast array of features.

Another key aspect is the relative ease of integrating these proxies into the broader system architecture. This integration streamlines the implementation process and reduces the time and resources required for development and testing. Additionally, the widespread adoption of popular proxy solutions facilitates their integration with other technologies, further enhancing their use.

Continuing our discussion on the underlying technology, utilizing one's own proxy technology can offer different benefits, similar to those seen in API gateways. This approach gives developers more control over the implementation, allowing the overall solution to address specific requirements that the service mesh provider has, such as optimizing for performance or simplifying the sidecars.

An example used in the Literature Review section for this approach is Linkerd, which employs the Rust language-based linkerd2-proxy as its underlying proxy technology. By deciding to develop its own proxy solution, Linkerd can offer a more performance-focused and simplified approach to sidecars than the Envoy proxy allows. The design of linkerd2-proxy, leveraging Rust's efficiency and safety features, demonstrates how a custom-built proxy can substantially enhance certain aspects of a service mesh, making it more suitable for specific operational needs.

The central point here is that while utilizing existing proxies like Envoy is a viable and

effective option, creating an in-house proxy mechanism can bring unique advantages to a service mesh implementation. Developing a custom proxy allows for a higher degree of customization and optimization, enabling the service mesh to align more closely with specific organizational goals or technical requirements.

There is a noticeable contrast between service meshes and API gateways in terms of the depth of the underlying technologies they utilize. The distinction in service meshes lies not only in the choice of proxy but also in the extent to which these proxies can function, accommodating a range of computing environments in which the service meshes operate.

An example of the diversity in service mesh implementations is Cilium Service Mesh, which I have discussed in this thesis. Unlike API gateways that operate in user space, Cilium functions directly in kernel space using eBPF. This shift to kernel-level operation marks a significant departure from the traditional user-space implementations common in most service meshes and API gateways. By leveraging the capabilities of kernel space, Cilium provides enhanced performance that is typically unattainable with solutions based on user space.

Cilium's decision to implement service mesh functionalities in kernel space presents a unique set of advantages and challenges. Kernel space allows for more efficient packet routing, improved observability, and lower latency communication between services. However, the kernel environment's limitations constrain this approach, necessitating a dedicated layer 7 proxy for more complex operations.

The differences in the underlying technological approaches are summarized in Table 3. In summary, organizations need to consider the technology behind the service mesh implementation to understand its benefits and challenges fully. This enables organizations to make decisions that are most beneficial to them.

Contrasting service mesh patterns

In the evolving landscape of service meshes, understanding the differences between service mesh architectures is crucial for organizations that wish to optimize their communication and service management. In the preceding sections, I have introduced service mesh patterns with examples, focusing on two dominant models in the service mesh landscape: sidecar architecture and sidecarless architecture. Each of these patterns offers distinct advantages and challenges, shaped by their unique operational dynamics.

This section aims to gather insights from our previous discussions and compare them more thoroughly. By contrasting the differences between the sidecar and sidecarless approaches, I intend to provide a more comprehensive viewpoint that will assist organizations in

Table 3: Comparison of service mesh tehcnology choices

	Existing Proxy Solution (e.g. Envoy)	**Own Proxy Solution (e.g. linkerd2-proxy)**	**Kernel-Level Service Mesh (e.g. Cilium)**
Benefits	Ease of adoption, proven performance, extensive support	Greater control, performance optimization	Efficient packet routing, improved observability, lower-level communication
Suitability for Operational Need	Broad applicability, versatile features	Tailored to specific operational requirements	High performance, suitable for intensive workloads
Challenges	Limited by proxy's inherent capabilities	Requires in-depth development and maintenance	Constrained by kernel environment limitations, needs its own proxy for advanced features
Example usage	Istio using Envoy sidecar proxies	Linkerd using Rust-based linkerd2-proxy	Cilium operating in kernel-space using eBPF
Performance Characteristics	Generally high, dependent on proxy	Dependent on custom implementation quality	Very high, direct kernel interaction
Alignment with Organizational Goals	Good for general purpose	Great for more specific organizational needs and goals	Ideal for scenarios requiring high performance and low latency

making more informed decisions.

There are multiple benefits to choosing the sidecar pattern over other service mesh patterns. One of the most notable advantages of this pattern is the security it offers. This is achieved by isolating the networking functions for each service, which effectively separates the communication channels, adding a layer of protection between services.

Another strong point of the sidecar pattern is the fine-grained traffic control it offers. The individual proxies for each service allow for more control over the management of each service proxy in the service mesh. Administrators can manage and tweak the network flow at a more granular level, accommodating the specific needs of each service in the cluster.

This independence also allows the sidecar pattern to have independent service management. This means that the pattern allows for independent configuration and management of each service. This independence is crucial for large-scale systems where services need to be managed individually.

Despite its benefits, the sidecar pattern also presents certain challenges. Firstly, it introduces noticeable latency and increased resource consumption in service-to-service communication (Zhu et al., 2023). This occurs because all traffic is routed through the sidecar proxies. Furthermore, these additional proxies consume more system resources compared to solutions without sidecars (Zhu et al., 2023), which is a significant factor in resource-constrained environments.

Additionally, the sidecar pattern increases the operational complexity of the service mesh due to the multitude of proxies involved(Duarte Maia and Figueiredo Correia, 2022). Managing a set of sidecar proxies adds a layer of complexity to operations, as each service's proxy needs to be configured, monitored, and maintained. This can become especially challenging in large-scale deployments.

While the sidecar pattern offers numerous benefits to the service mesh, it can also impact performance. This performance penalty arises from the extra hops in network communication. When two services communicate, the communication must pass through both corresponding proxies. This additional step, due to the sidecar proxy, affects the overall performance of the system, particularly in scenarios requiring high throughput.

As the sidecar pattern introduces a significant performance penalty and consumes more resources, new methods for handling service-to-service communication have been developed. The sidecarless pattern is exemplified in the implementations of Istio's ambient mesh and Cilium service mesh.

Unlike the sidecar pattern, the sidecarless approach does not create a sidecar container for each pod. Instead, it manages communication without sidecars. This can be

achieved in various ways, such as through the node agent pattern (Duarte Maia and Figueiredo Correia, 2022). In this pattern, a single proxy is created per node, handling all traffic within that node. This is in contrast to the sidecar pattern, which employs individual proxies for each pod.

Additionally, there are other approaches to the sidecarless pattern beyond the node agent. For example, Istio's ambient mesh employs a strategy where proxies are created for each service rather than each instance. This means that multiple instances of the same service can share a single proxy, reducing the total number of proxies required in highly scaled microservice environments.

The sidecarless pattern in service mesh architectures offers distinct advantages, primarily stemming from the reduced number of proxies required. This characteristic significantly impacts the service mesh's operational complexity and resource utilization.

1. **Reduced Operational Complexity**: One of the most notable benefits of the sidecarless pattern is the decrease in operational complexity associated with managing service meshes. In a microservice architecture that adopts this pattern, the number of proxies required is considerably smaller, as there is no need for individual sidecars in each pod. Consequently, administrators are tasked with managing a relatively small number of proxies, regardless of the size of the service network. The sidecarless approach simplifies the maintenance and configuration of the service mesh.

2. **Lower Resource and Memory Usage**: Each proxy in a service mesh consumes a certain amount of system resources and memory. By minimizing the number of required proxies, the sidecarless pattern inherently reduces the overall resource and memory consumption in the cluster. This aspect becomes critically important in systems hosting a large number of services, where the resource demand for service proxies could be substantial. In such scenarios, the sidecarless pattern offers an efficient solution, mitigating the potential strain on CPU and memory resources.

The sidecarless pattern offers a more simplistic and resource-efficient approach to service-to-service communication within service mesh architectures. However, this approach is not without its challenges. In scenarios where the shared proxy encounters issues, such as a configuration error or malfunction, the impact, often referred to as the "blast radius", is increased. A single fault in the proxy can simultaneously affect multiple pods within the cluster.

Additionally, the sidecarless pattern presents limitations in resource usage granularity. Unlike the sidecar architecture, where there is enhanced control over individual pods and their respective proxies, achieving this level of detailed management is more challenging

in the sidecarless model. Here, multiple pods rely on a common proxy for communication, making it difficult to fine-tune resource allocation and settings for each pod.

Lastly, fairness concerns arise within the sidecarless pattern. This relates to the allocation of the proxy's resources among various services. The shared nature of the proxy necessitates a strategy for effectively managing and prioritizing traffic from multiple services. Without this mechanism, a single service might monopolize the proxy's resources, thereby affecting the performance of other services dependent on the same proxy.

In conclusion, Table 4 depicts the essential contrasts between the sidecar and sidecarless service mesh patterns. It illustrates the trade-offs inherent to each approach. By comparing these patterns side-by-side, we gain a clearer understanding of their respective strengths and limitations. This comparative analysis serves as a valuable guide for organizations in selecting the most suitable service mesh architecture for their specific needs. Ultimately, the choice between sidecar and sidecarless patterns should be informed by the unique requirements and constraints of the given operational environment.

Table 4: Service Mesh Pattern Comparison

	Sidecar pattern	**Sidecarless pattern**
Security	Enhanced through own proxy layer	Less granular security options
Traffic Control	Fine-grained, with individual proxies for each service	Less granular in traffic control due to shared proxies
Operational Complexity	Higher, due to the need to manage numerous proxies.	Reduced, with fewer proxies to manage.
Resource Usage	Increased, as each service requires its own proxy.	Lower, due to fewer required proxies
Performance Impact	Noticeable latency and resource use due to additional proxy hops.	More efficient, but challenges in resource allocation fairness.
Independence in Management	High, allowing for independent configuration of each service.	Limited, as multiple services share the same proxy.
Scalability challenges	Managing a large number of proxies can be complex.	Easier scalability with fewer proxies, but potential resource contention.

4.1.3 Conclusion of Similar Solutions

This section of the thesis has highlighted the differences between similar solutions in microservice networking. While API gateways primarily focus on north-south traffic and service meshes on east-west traffic, the capabilities each offers can vary significantly.

There are several factors to consider when choosing between similar solutions for both API gateways and service meshes. API gateway vendors differentiate themselves based on the extent of integration with API management solutions and the choice of internal technology. In contrast, service meshes vary according to the patterns employed for service-to-service communication and the specific underlying technology used to manage traffic.

As a result, organizations face a complex decision-making process when selecting among different solutions. Given the distinct differences in their approaches, capabilities, and intended traffic management directions, there is no one-size-fits-all solution. The ”best” choice is highly subjective and depends on the specific requirements, goals, and existing infrastructure of the organization.

It is crucial for organizations to thoroughly assess their own needs and understand the trade-offs involved with each approach. Factors such as the desired level of security, scalability, ease of maintenance, and the nature of the network traffic play pivotal roles in this decision. Additionally, the integration capabilities with existing systems and the organization's technical expertise are key considerations.

However, the overlapping capabilities of service meshes and API gateways allow for a comparison between these solutions. In the next section, I will compare the solutions with each other, thus making the choice between different approaches easier for an organization's needs.

4.2 Contrasting Different Solutions

I have already discussed various solutions for handling traffic in Kubernetes clusters. However, there has not been a detailed discussion on how these solutions compare to each other. To determine the best fit for their needs, organizations must understand the unique differences these solutions offer and how each relates to the overall system. In this section, I will explore the convergence of service meshes and API gateways, and what this implies for broader Kubernetes networking.

As we navigate through this section, readers should anticipate a shift in perspective from traditional terminologies to a broader viewpoint. Instead of focusing solely on API gateways, I will explore the subject from the viewpoint of ”Application Networking”. This perspective not only offers a more comprehensive understanding of the topic but also

aligns with the evolving nature of these technologies, which are increasingly overlapping in capabilities.

As a result, even though service meshes and API gateways are conceptually distinct, comparing them is beneficial for gaining a deeper understanding of the topic. The focus is not on the naming of these solutions, but on the problems they can solve for an organization.

After all, the primary goal for organizations is not merely to use an API gateway or service mesh, but rather to leverage them in achieving other objectives. Both solutions can bring significant benefits to the overall system, and they can even be used together. For these solutions to complement each other effectively, a well-conceived integration strategy is essential. While this section aims to highlight the similarities between these solutions, it's important to recognize that each brings its unique benefits. Consequently, there is no one-size-fits-all solution that encompasses all the capabilities these technologies offer.

Differences between solutions

While API gateways and service meshes are distinct technologies, they can easily be confused, particularly in discussions about modern cloud-native API gateways and service meshes within microservice architectures. To provide a clearer understanding, it is beneficial to address the potentially confusing aspects of these solutions initially. This approach ensures that later discussions do not mix up these distinct concepts.

1. **Possibility of Similar Underlying Technology**: Though API gateways and service meshes are distinct concepts, they can employ similar underlying technologies. For instance, the Envoy proxy can be used both in service mesh and API gateway implementations. However, the actual implementation can vary significantly, such as employing the sidecar pattern for service meshes and a single proxy layer for API gateways.

2. **Functionality Convergence**: As discussed later in this section, and as evident from earlier sections, both solutions are capable of handling observability and security within microservice networking. This functionality convergence is a notable aspect of their evolving roles.

3. **Both Can Create a Facade for North/South Traffic**: While traditionally service meshes have focused more on service-to-service communication, they are also capable of creating an API gateway for the cluster. This role was previously fulfilled by Kubernetes Ingress, which manages north/south traffic.

Although there are similarities between the concepts, we can identify distinct viewpoints

to differentiate these approaches. However, these viewpoints are not without counterarguments, meaning there is still no definitive way to distinguish the terms in all cases:

1. **North/South and East/West Viewpoint**: Traditionally, API gateways can be distinguished from service meshes by comparing their primary communication participants. In an API gateway, the communication is typically between the client and the service, whereas in a service mesh, it is between the services themselves. The key distinction lies in the fact that clients, unlike internal services (microservices), are not elements that can be controlled by the system. Therefore, a service mesh primarily facilitates east/west traffic, in contrast to the north/south orientation of API gateways. However, it's important to note that service meshes can also handle north/south traffic, as evidenced by the use of Kubernetes Ingress and Gateway API.

2. **Business Logic versus Service Communication**: Another perspective for differentiating these solutions focuses on their primary areas of application. Given their position at the edge of the system, API gateways often serve to create APIs-as-Products capabilities. This positioning means that incoming communication to the system typically passes through the API gateway, making it a suitable place for API management functionalities. Conversely, service meshes are more adept at handling efficient service communication logic. While this distinction is generally applicable, it is not absolute. For example, some API management capabilities, such as developer portals, can also be implemented at the service mesh level (Solo.io, 2020).

3. **Convergence in Features**: Finally, we can compare API gateways and service meshes based on the features they provide. Both service meshes and API gateways offer functionalities related to service connectivity, such as handling L4 and L7 networking. However, it's important to discern the ideal management location for each of these converging capabilities, as one solution may be a more suitable choice for an organization depending on the context. Although there is a convergence in feature sets, it is also crucial to recognize that even similar solutions can have significantly different feature sets. A more in-depth discussion of these converging features will follow later in this section.

Using these viewpoints, we can conceptually distinguish service meshes from API gateways. However, this distinction does not preclude the situation where a service mesh also implements API gateway functionalities as part of its implementation. The conceptual foundation rests on the fact that the focus of API gateways is on the boundaries of the system. These boundaries are not limited to just the interface between the cluster and the cluster's internal services; they can also exist between different sets of

services within a cluster. For instance, a company might have multiple products within the same cluster. In such a case, an API gateway could be implemented inside the cluster to manage the communication between these products, even if the communication never leaves the cluster.

The focus areas of these approaches also differ. Service meshes excel in inter-service communication, where interactions are complex and require efficient and reliable management. This focus is evident from their design and the features they offer, such as load balancing and service discovery.

On the other hand, API gateways are often viewed through the lens of a centralized approach. They act as the entry point to the system, effectively managing and controlling access to various services. This centralization simplifies the system's interface, making the set of services appear as a unified unit.

In contrast, a service mesh adopts a more decentralized approach, evident in patterns like the sidecar model. Here, each service is paired with a proxy, decentralizing network functionality across all internal services. This decentralization allows for more granular control and customization at the service level.

Similarities between approaches

While the feature sets can differ even among similar solutions, we can still compare these different solutions to each other by focusing on a high-level overview, without considering specific vendors. As a result, the discussion here aims to provide an overview of the similarities between solutions. However, a more in-depth comparison should be undertaken by organizations wishing to compare different vendors against each other.

When discussing service mesh capabilities from a high-level perspective, we can identify distinct feature areas. First and foremost are the service connectivity features. These include functionalities related to how services connect to the overall system, such as load balancing, service discovery, traffic routing rules, and observability. Additionally, service meshes provide Layer 4 features.

Service meshes operate on a broad spectrum, addressing service connectivity challenges that extend beyond just L7 traffic. While they do manage HTTP traffic, their functionality is not limited to this alone. Service meshes are designed to handle a variety of traffic types, including lower-level protocols. This capability is crucial because service connectivity involves more than just HTTP traffic. Consequently, service meshes tend to focus more on L4 traffic compared to API gateway solutions. This focus ensures efficient and reliable communication between services. The emphasis on the L4 layer distinguishes service meshes from API gateways and highlights the respective roles of each solution in

a microservices environment.

API gateways also encompass service connectivity as one of their important capabilities, primarily focused on L7 due to their unique position as the boundary between systems. The capabilities of API gateways extend to include potential API management features and the concept of APIs-as-Products. Both of these aspects contribute to providing a uniform API interface for the product. This involves bundling multiple API endpoints from different services to create a cohesive API that can function as a standalone product. By creating a facade in front of the system, API gateways effectively decouple the client from the internal services of the system, thereby simplifying service usage for external users.

From this high-level overview of the capabilities of service meshes and API gateways, we observe that both solutions are equipped to handle the service connectivity features of a system. These features include routing, tracing, load balancing, logging, and others related to the connectivity of services. Consequently, there is an overlap in the feature sets offered by both solutions.

It should be noted that while features are converging, the application and impact of these features can significantly differ between the two solutions. The features listed here are not exhaustive but highlight the differences in approaches.

API Gateway focused features:

API-as-Product features such as request aggregation API Gateways excel in presenting APIs as products because they serve as the system's entry point. This role involves not just exposing services but managing them effectively. For instance, in request aggregation, multiple requests are combined into a single request, which is then sent back to the client.

External authentication and authorization providers As the sole entry point, API Gateways can more easily connect to external authentication and authorization providers, enhancing the security features of the API gateway. For instance, they can utilize OIDC and OAuth as external identity providers.

These features underscore the necessity for positioning the API gateway at the system's boundary, as it needs to handle data coming from clients outside the internal services.

Service mesh focused features:

Layer 4 management Service meshes, as previously noted, operate at lower levels due to service networking often occurring at these levels. While L4 traffic is not exclusive to service meshes, it is more prominent in service mesh implementations than in API gateways, owing to differences in communication parties.

Resiliency features, such as circuit-breaking Service meshes also excel in features that enhance the overall resilience of the system. An example is the circuit breaking pattern, which aids in detecting system failures.

In contrast to the API gateway, these service mesh features need to occur during service-to-service communication. The API gateway is involved only in requests at the system's boundary, whereas these features operate within the system. Consequently, service meshes possess unique features tailored to service-to-service communication.

Features Common to Both API Gateways and Service Meshes:

Rate throttling Both API gateways and service meshes offer rate throttling or limiting capabilities, essential for managing traffic flow and preventing system overload. The focus, however, differs: service meshes can granularly limit rates for services, whereas API gateways more effectively limit client communication.

Layer 7 features Both solutions provide functionalities at the application layer, handling HTTP/HTTPS traffic and other high-level protocol communications.

Authorization and authentication These are key features in both solutions. However, their approaches vary: API gateways typically handle these for external, unknown clients and can connect to external authentication and authorization providers such as OpenID Connect (OpenID Foundation, 2023), while service meshes manage them within the realm of known, internal entities.

Security Both API gateways and service meshes enable robust security measures to protect the data flowing through the system.

Policy enforcement Policy management and enforcement are integral to both technologies, allowing administrators to define and apply rules governing traffic within the system.

Observability This is a shared feature, but the depth and scope differ. API gateways track and monitor the traffic they process but lack insights into deeper service-to-service interactions. In contrast, service meshes provide a more comprehensive view, offering observability across the entire network of internal service communications.

This is not a definitive list of all the converging features, but it suffices to showcase that some features can be implemented both at the boundary of the system and within the system. However, as these examples illustrate, there are still some differences between similar features. For instance, the handling of authentication and authorization with external providers, such as OpenID and OAuth providers, in API gateways.

The convergence of capabilities creates an important discussion regarding the use-cases of both API gateways and service meshes. While service meshes are becoming more sophisticated, they do not entirely replace API gateways. Some features are more effectively handled at the service mesh layer due to enhanced performance, but service meshes lack the ability to manage all the features that API gateways can at the system's edge. Most notably, API management features are more challenging for service meshes to implement, as these are designed for north/south traffic. For instance, usage/policy quotas, self-service/developer portals, analytics, and monetization are all oriented towards traffic entering the system. Therefore, if an organization requires both API management and service-to-service observability, it is advantageous to use both solutions simultaneously. This approach allows for the optimization of both east/west and north/south traffic.

Modern service mesh solutions are increasingly emphasizing north/south traffic, with continual evolution in their north/south capabilities. Istio, one of the most popular service mesh implementations, already includes a north/south solution, and expanding features to handle north/south traffic is also on Linkerd's roadmap (Linkerd, 2023a). As technology continues to evolve, causing these features to converge, organizations find selecting the best solution increasingly challenging. While the north/south capabilities of service meshes are still developing and not yet on par with API gateways, there is also a growing focus on integrating service mesh solutions with API gateway solutions.

4.3 Impact of Standardization

Up to this point, we have focused on the evolution and the current state of Kubernetes networking. Yet, emerging trends are significantly shaping the future of Kubernetes networking solutions. A central theme in this thesis is the movement toward the standardization of these solutions. This section will delve deeper into the standardization process within Kubernetes networking, examining its implications and the transformative potential it holds.

The discussion on standardization in this thesis is centered around the Kubernetes environment, given its relevance to Cloud Native API gateways and service meshes. However, it's crucial to acknowledge that API gateway solutions also exist outside Kubernetes clusters. Within Kubernetes, solutions such as API management platforms are less standardized due to their specialized roles in the system. These platforms, which offer API gateways, typically market themselves as complete products, thereby reducing the emphasis on interoperability compared to more commoditized API gateways. Despite this lower degree of standardization, API management platforms remain popular and can be an excellent choice for organizations seeking a comprehensive API management

solution from a single product.

While platform API gateways have not experienced significant standardization, Kubernetes-focused API gateways and networking solutions are increasingly embracing standardization. This shift is beneficial, as it allows users to adopt different API gateways more seamlessly if the configuration is consistent across various solutions. It also simplifies the process of switching between solutions when they closely resemble each other. An example of this trend is the use of the Kubernetes Gateway API to standardize the configuration of API gateway implementations.

Kubernetes networking solutions are standardizing by shifting from the Ingress API to the Kubernetes Gateway API and preferring well-tested proxies as their underlying technology. Initially, Kubernetes networking primarily utilized the Ingress API for managing north/south traffic within clusters. However, with the growing need for more sophisticated configuration methods, the Kubernetes Gateway API was introduced, enhancing capabilities and offering greater flexibility. This shift underscores the ecosystem's progression toward more advanced networking solutions. Additionally, the widespread adoption of well-tested proxy technologies, noted for their ease of use and reliable performance, have become a popular choice for many implementations.

4.3.1 Standardization of Configuration

The Kubernetes Gateway API, designed with extensibility in mind (Kubernetes Gateway API, 2023d), represents a considerable improvement over the previous Ingress API. While the Ingress API remains in use, the expanded capabilities of the Gateway API offer a more flexible solution for API gateway and service mesh providers. Due to its greater flexibility, the Gateway API has also been employed to configure popular service mesh implementations such as Istio and Linkerd (Kubernetes Gateway API, 2023b). Consequently, the Kubernetes Gateway API has become a universal specification for networking in Kubernetes, thereby standardizing networking configuration within the Kubernetes ecosystem.

A key aspect of the Kubernetes Gateway API's contribution to the standardization of Kubernetes networking lies in its facilitation of an extensive and role-oriented API. Both service mesh and API gateway solutions can be configured using similar Kubernetes resources as defined by the Gateway API, such as Gateway and Route resources. This uniformity in configuration resources significantly eases the adoption process.

By employing a consistent set of resources across different networking solutions, the Kubernetes Gateway API effectively streamlines the networking configuration process. For instance, whether an organization is deploying an API gateway, a service mesh, or both, the approach and specification remain largely the same. This consistency is not

just a matter of convenience; it represents a shift towards a more standardized approach to Kubernetes network management.

Furthermore, the standardized specification across both API gateways and service meshes has implications for end users of these solutions. It reduces the learning curve for developers and system administrators, who can now apply their knowledge of these resources across various networking tools within the Kubernetes ecosystem. This interchangeability of skills and configurations ensures that organizations can more easily adopt and integrate different solutions into their own Kubernetes systems.

Therefore, the Gateway API stands as a unifying force in the Kubernetes networking landscape. The standardization of configuration resources simplifies the management of complex networking tasks and facilitates the switching between different API gateways or service meshes.

The Kubernetes Ingress resource, while serving as a basic solution for managing north/south traffic in Kubernetes clusters, exhibits limited flexibility compared to the Gateway API. This limitation stems from the inherent design of the Ingress resource itself.

A key factor that restricts the flexibility of Kubernetes Ingress is its definition as a singular resource within Kubernetes. This singular configuration approach contrasts with the layered approach of the Gateway API. For example, the Gateway API allows various resources to be managed by different roles within an organization. Application developers might handle Route resources, while system administrators configure cluster-wide settings at the Gateway resource level. This multi-layered approach provides greater customization and control than the singular configuration method of the Ingress API offers.

Moreover, due to this singular approach, Ingress is inherently limited in the scope of features it can provide. To overcome these limitations and enable more advanced configurations, Ingress controllers often rely on annotations or custom resources. Consider the following Ingress configuration as an example:

Listing 1: Example of an Ingress annotation configuration in Kubernetes using NGINX

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: annotation-example
  annotations:
    nginx.org/client-max-body-size: "2m"
spec:
  rules:
  - host: test.site.com
```

```
http:
  paths:
  - path: /foo
    pathType: Prefix
    backend:
      service:
        name: foo-svc
        port:
          number: 80
  - path: /bar
    pathType: Prefix
    backend:
      service:
        name: bar-svc
        port:
          number: 80
```

This configuration demonstrates how annotations are used to enhance the basic capabilities of the Ingress resource. However, this approach introduces challenges in standardization, as it depends heavily on controller-specific extensions.

The dependence on annotations and custom resources results in a lack of standardization across different Ingress controller implementations. The unique configuration approach of each controller leads to inconsistencies between various Ingress controllers. Therefore, there is no guarantee that the custom resources of one Ingress controller will be similar to those of another. This discrepancy makes it difficult for users to transition between different solutions within the Ingress framework, implying that custom resources and annotations in Ingress are not portable between different implementations. An annotation or custom resource that is specific to one Ingress controller might be non-existent in another, complicating the migration process and reducing overall system flexibility.

In summary, while Kubernetes Ingress provides a foundational framework for managing north/south traffic in Kubernetes, its design has inherent limitations that affect the portability and flexibility of the Ingress specification. These limitations highlight the need for solutions like the Kubernetes Gateway API, which aims to standardize and enhance the network configuration capabilities within Kubernetes environments.

4.3.2 Technology Standardization

Another perspective on standardization in Kubernetes networking involves the previously mentioned concept of similar underlying technology choices across different solutions. Utilizing ready-made proxies, such as Envoy, offers numerous benefits for various implementations, as previously discussed. This notable trend contributes to the

standardization of different implementations that share the same underlying technology, reflecting a broader movement toward uniformity and interoperability in Kubernetes networking.

Envoy Proxy stands out among various proxy technologies due to its widespread use in popular implementations like the Istio service mesh. Its adoption across a diverse range of applications underscores its versatility and robustness. This has led to a situation where many common Kubernetes networking solutions are based on similar technologies. These shared technologies have significant implications for how solutions are designed, implemented, and maintained.

The adoption of common technologies like Envoy Proxy in different networking solutions standardizes certain aspects of their design and functionality. This standardization can result in similarities in feature sets and operational capabilities. Furthermore, it enhances interoperability between service meshes and API gateways, as they share a common underlying technology.

Considering the choice of technology upon which a solution is built can also extend to higher levels, as demonstrated by Envoy Gateway. Flexible proxy technologies like Envoy Proxy, though powerful, has often been challenging to configure and use. Envoy Gateway has focused on simplifying this process, making Envoy proxies more accessible to developers and system architects. This simplification enables them to leverage Envoy's capabilities without the steep learning curve typically associated with the proxy. Such ease of use not only opens the door to advanced networking features but also accelerates the deployment and management of networking functionalities.

The integration of Envoy Gateway with Kubernetes, particularly through its implementation of the Kubernetes Gateway API, is a crucial aspect of its functionality. The incorporation of the Gateway API demonstrates its alignment with Kubernetes' native network management, enhancing its compatibility within Kubernetes clusters. This integration broadens its adoption, underscoring its role in the standardization of API gateways within the Cloud Native ecosystem.

Envoy Gateway's approach is to provide basic Envoy proxy features through an easier-to-adopt API, thereby facilitating a smoother adoption process for end-users. By offering a unified core API, Envoy Gateway eliminates the need for vendors to duplicate basic features, enabling them to concentrate on developing unique aspects of their offerings. This move towards standardization creates an environment where new solutions can be built upon a reliable and standardized base for API gateways.

5 Discussion and conclusion

This discussion section seeks to reflect on Kubernetes networking by answering the research questions presented at the start of the thesis. The thesis answers three distinct questions about Kubernetes networking: the evolution of API gateways, available service networking approaches in cloud-native architectures, and the relative strengths and weaknesses of these approaches. The preceding sections have systematically presented evidence and analysis addressing these questions. Now, this section will summarize these findings, offering a concise overview and practical insights for Kubernetes networking.

5.1 Summary of Findings

Discussion on the evolution of API gateways demonstrates their significant technological advancements, influenced by changing application requirements and their impact on application architecture. The evolution of architectures introduced new methods for managing north/south traffic, as evidenced by current API gateway solutions. Traditional platform API gateways focused on integrating API management, whereas commoditized cloud-native API gateways emphasized networking functionality and integration with highly dynamic systems. These cloud-native gateways were designed as interchangeable components within the broader system. This commoditized approach underscores the importance of interoperability for API gateways and the trend towards decoupling API management from the gateways' basic functionalities. This evolution reflects the industry's move towards more flexible and modular solutions to effectively manage north/south traffic.

Kubernetes networking encompasses varied traffic management approaches. Each solution has its specific focus within networking, yet their offered features tend to converge. The direction of the traffic is an important factor distinguishing each approach. Solutions focused on north/south traffic in Kubernetes networking include API gateways and the simpler north/south-focused technology, Ingress controllers. Conversely, east/west traffic can be efficiently managed with solutions specifically designed for service-to-service communication, such as service meshes. Furthermore, the discussion on service meshes revealed that while focus areas might differ, solutions for both north/south and east/west traffic often handle similar features.

Research question 3 delved into the strengths and weaknesses of various Kubernetes networking solutions. The discussion showed that the direction of the targeted traffic affects each approach's relative strengths and weaknesses. API gateways, focusing on client-to-service communication, offer system boundary control features such as API management capabilities. Another strength of API gateways is their ease of use and

maintenance. The centralized deployment of the gateways positions them at the system's edge and keeps the number of API gateway proxies relatively small. Although API gateways excel in managing communication at the system's edge, they also have notable weaknesses. Primarily, API gateways lack the needed visibility in service-to-service communication. When traffic bypasses the API gateways, as it does in intra-cluster communication, the visibility provided by the API gateway is more limited than what a service mesh can offer. Furthermore, as distinct parts of the system, API gateways introduce an additional network hop, thereby increasing request times due to the extra hops. These limitations were further discussed in the thesis.

Ingress controllers are simpler proxies than API gateways for handling north/south traffic in Kubernetes clusters. Consequently, they share similar strengths and weaknesses with API gateways. However, Ingress controllers are more limited in their capabilities compared to API gateways. The discussion in the thesis did not attempt to distinguish between API gateways and Ingress controllers due to the similarities in both their focus and the features they offer.

The strengths and weaknesses of east/west traffic-focused service meshes were discussed in the thesis, introducing various providers with different emphases on service-to-service communication. When examining service meshes as a whole, the thesis noted distinct features that service meshes excel in. One fundamental strength of service meshes is the granular control and observability they offer. The granularity of control over the proxies depends on the service mesh architecture, but it is generally more refined compared to centralized API gateways. Secondly, service meshes can easily implement system-wide features. Architectures such as the sidecar model can add security features to each service in the cluster, thus reducing the need to implement these features within the services themselves. However, the large number of proxies required by service meshes consequently increases their complexity and the configuration needed. The focus is also on service-to-service communication, meaning the capabilities offered for client-to-service traffic are limited.

Analyzing the evolution of API gateways clarifies that the industry's varied collection of API gateways responds to changing application architectures. This has led to distinct differentiations among API gateways. The research in the thesis highlights the divergence in API gateways' approaches to API management. Initially, the market preferred API gateways that included integrated API management features. However, the industry is transforming. A trend is emerging to decouple traditional API management features from the essential functions of API gateways. This evolution is fostering the development of more modular and flexible API gateways, specifically tailored for dynamic environments like Kubernetes. Consequently, the use of modular design in cloud-native architecture meets modern application requirements such as reliability and scalability. While not a

novel concept, modern API gateways, as the thesis discussion reveals, are equipped to meet the cloud-native landscape's demands.

5.2 Organizational Impact and Strategic Considerations

In addressing Research Question 2, this thesis has comprehensively explored Kubernetes networking approaches, specifically focusing on API gateways and service meshes in cloud-native environments. These solutions serve as viable strategies to meet various organizational needs in network management. The problems that cloud-native applications need to face in networking were mentioned in the cloud-native introduction, which includes service discovery, security concerns, and observability and monitoring. The aforementioned networking solutions can handle each of these challenges to varying degrees, as the thesis showcased in its discussion.

The thesis specifically addressed the observability and monitoring challenges in Kubernetes networking within the service mesh and API gateway contexts. While both approaches provide observability and monitoring capabilities, they differ in focus, as detailed in the comparison section. API gateways primarily offer insights into client-to-service communication, whereas service meshes provide a comprehensive view of all communications within the system. As integral parts of all services in the network, service meshes also deliver more detailed observability than API gateways. Understanding the distinct observability features of service meshes and API gateways is crucial for organizations to choose the right tool, particularly if their primary concern is enhancing the observability or monitoring of services.

Regarding service discovery in Kubernetes networking, both the API gateway and service meshes employ Kubernetes' native service discovery mechanism to monitor services within the cluster. By understanding these services, these approaches can leverage this knowledge to develop additional features, such as various routing mechanisms. The thesis findings indicate that these approaches showcase differences in their capabilities, with such variations being inherent to the focus areas of the respective approaches, such as layer 4 focused features in service meshes.

Organizations can also focus on their services' reliability, thus considering integrating a service mesh or API gateway into their system. Both the API gateways and service meshes are capable of providing mechanisms for reliability features, such as circuit-breaking, and timeouts. As the feature set in the approaches converged in the networking features, reliable communication can happen at the both API gateway level and service networking level with the service meshes.

Both API gateways and service mesh implementations consider security as one of their core functionalities. The thesis discussed the differences in security between client-

to-service and service-to-service communications. The findings indicate that while both approaches prioritize security features, they differ in how they establish secure communications. The discussion highlighted that the variance in security features, such as authentication and authorization, stems from the focus on either north/south or east/west traffic. In the context of north/south traffic, API gateways manage security measures for users accessing the services, including integration with external security providers for authentication and authorization. In contrast, service meshes handle service-to-service communication, which often occurs below the application layer. In these scenarios, features like mutual TLS are emphasized.

The primary advantage of implementing a Kubernetes networking solution lies in its ability to facilitate efficient service communication and connectivity. This efficiency is evident in both API gateway and service mesh contexts. API gateways are specifically designed to streamline communication between clients and services. Positioning an API gateway within the edge of the system not only improves interactions between clients and services but also enables the integration of additional features into these communications. The API gateway's features, including load balancing and routing mechanisms, exemplify the additional functionalities tailored specifically to enhance communication efficiency. These elements stand out as notable examples within the wider scope of improving service communication between clients and servers.

When it comes to service-to-service communication, the comparison section of the thesis demonstrated that service meshes are more efficient than API gateways. This is primarily because service meshes do not require extra network hops for communication, unlike in the API gateway context. The thesis explored various approaches to efficient service communication within Kubernetes clusters. Each service mesh employed its unique methods for managing communication, such as sidecar and sidecarless architectures. Therefore, the choice of service mesh significantly affected service communication performance. For instance, using Cilium for low-level communication proved to be beneficial in terms of performance.

In summarizing the findings related to research question 2, this thesis has explored the varied capabilities and strategic roles of Kubernetes networking solutions such as API gateways and service meshes in cloud-native environments. For organizations invested in cloud-native applications, these solutions addressed the challenges posed by cloud-native architecture's dynamic nature.

The comparative study of API gateways and service meshes highlighted their respective strengths and weaknesses in managing specific aspects of Kubernetes networking. The findings show that API gateways excel in managing client-to-service communication, acting as a crucial access point for clients.

On the other hand, service meshes are more tailored to service-to-service communication within the Kubernetes environment, and they have increasingly acquired capabilities for handling north/south traffic in recent years. The chosen solution's architecture significantly impacts the service mesh implementation, affecting its feature range and overall efficiency.

Ultimately, an organization should align its Kubernetes networking solution selection with its unique needs and priorities. Whether the focus is on external client interactions, internal service communications, or a combination of both, the networking solution must meet the organization's specific requirements. The findings of this thesis underscore the importance of a careful and informed selection process when choosing a networking solution for an organization.

5.3 Implications of Research Findings

The convergence of API gateways and service mesh within Kubernetes environments has wide-ranging implications for organizations and the cloud-native landscape. The findings in this thesis not only illustrate the current state of Kubernetes networking but also project future trends within the cloud-native landscape.

The increasingly blurred line between service meshes and API gateways creates one key impact. As the capabilities of both solutions converge further, they can address a wider range of networking challenges in cloud-native environments. This is exemplified by the implementations mentioned in the thesis, such as the Istio service mesh, which now handles north/south traffic to some extent. Such developments indicate a trend towards more universal Kubernetes networking solutions, capable of handling a diverse set of networking problems in both internal and external cluster communications.

Furthermore, the findings highlight a significant shift towards increased flexibility and interoperability within the Kubernetes ecosystem. The adoption of standardized approaches across various implementations facilitates smoother transitions between different systems. Kubernetes itself has played a crucial role in this standardization by introducing new APIs for enhanced service networking and refining existing ones to better support current implementations, including the native support for sidecar containers. This drive for standardization is also evident among implementers who have rapidly adopted these new, Kubernetes-specific APIs for service networking.

The research findings on Kubernetes networking carry significant implications for organizations and practitioners planning to implement service networking solutions in their cloud-native architecture. These findings emphasize the importance of strategic decision-making in selecting the right solution for cloud-native architecture, whether it be an API gateway or a service mesh. The choice of implementation can profoundly

impact the organization.

Firstly, organizations must thoroughly evaluate both API gateways and service meshes to grasp their distinct functionalities and overlapping features. Understanding these distinctions is crucial to avoid implementing a solution that does not meet the organization's specific needs. For example, if the primary concern is inter-service communication, a service mesh might be a more suitable solution due to its superior performance. Conversely, an API gateway may be the better choice for managing external traffic and handling API management features, given API gateway providers' increased focus on these areas. The comparative section of the thesis highlights the subtle differences in similar features across these solutions.

Another consideration for organizations is adapting to standardization trends. By adopting standardized solutions, such as those conforming to the Gateway API, organizations can ensure compatibility and ease the transition between solutions. This alignment also simplifies the learning curve for development teams, as they need not master as many implementation-specific tools.

The evolution of Kubernetes networking solutions has resulted in a diverse range of networking options, each with its own focus areas and relative strengths. This variety presents an opportunity for organizations to tailor their systems by selecting solutions that best fit their unique requirements. Consequently, organizations can optimize their systems based on performance or other metrics, such as ease of use, by choosing the right combination of Kubernetes networking implementations.

The convergence of capabilities and ongoing standardization in Kubernetes networking necessitates operational flexibility from organizations. Decisions made today must consider not only current trends but also future adaptability and scalability. This requires staying up to date on advancements and being prepared to revise or update networking strategies as the cloud-native landscape continues to evolve.

In summary, the findings from this research have profound implications for organizations implementing Kubernetes networking solutions. Successfully navigating these implications requires understanding the available options and discerning their differences.

5.4 Implications for Vertex Systems

As this thesis concludes, it is essential to reflect on the specific impact of the findings on real organizations, such as Vertex Systems, introduced in the initial sections. The comprehensive evaluation of Kubernetes networking solutions, with a focus on the architectural differences between these solutions, lays the groundwork for strategic decision-making in organizations. Previous sections of this thesis discussed the preferable

networking solutions for organizations on a general architectural scale. For more in-depth comparisons between similar solutions, further analysis is required. This section demonstrates how the findings and their interpretations can be applied in the context of Vertex Systems and its infrastructure.

Throughout the thesis, various architectures of Kubernetes networking solutions have been introduced and examined in the context of cloud-native applications. This analysis highlighted their unique roles in networking and the relative strengths and weaknesses of each solution. This section aims to guide Vertex Systems in identifying the networking solution and architecture that best aligns with their current and future needs, considering the unique characteristics of their system.

5.4.1 Understanding Requirements for Networking Solutions

Vertex Systems' drive toward cloud-based products, especially the new Vertex Sync, underscores the need for robust and secure networking solutions. This section explores the key business requirements that any networking solutions must meet to support the evolving products of Vertex Systems.

Firstly, it's important to note that Vertex Systems handles sensitive CAD data from various clients, necessitating strict security measures. The networking solution must guarantee secure data transfer throughout the system, protecting against unauthorized access. This includes implementing data encryption and authentication protocols. Ensuring data integrity is not just a technical necessity but also crucial for maintaining client trust and compliance.

The launch of the new cloud application, Vertex Sync, is expected to significantly increase the volume and complexity of data flow within Vertex Systems. This underscores the need for enhanced observability and traceability of the data. A current challenge is the absence of a comprehensive mechanism to track client-service requests and responses. The networking solutions should provide a clear view of data flow, simplifying the debugging process in communication issues. Currently, debugging relies on time-consuming log reviews at the microservice level, which fails to provide a complete overview of situations. Effective observability and traceability are essential for troubleshooting, monitoring, and gaining insights into system performance and user behavior.

Vertex Systems also includes the potential monetization of its new APIs in its roadmap, underscoring the importance of a networking solution that can integrate with API monetization tools. The ideal networking solution should be flexible enough to support future monetization strategies, either by implementing these strategies themselves or by easily integrating with an API monetization platform. However, due to current uncertainties regarding the exact monetization model and the infrastructure required, this

aspect remains a consideration for future adoption rather than an immediate requirement for Vertex Systems' infrastructure.

When selecting a networking solution, it's crucial to understand the nature and characteristics of traffic within the organization's infrastructure. This insight will assist in choosing an ideal solution for the networking context, featuring capabilities that best support Vertex Systems' operational needs.

A key consideration for Vertex Systems is the fluctuating volume of data traversing the system. Vertex Sync, which handles CAD data, often necessitates transferring large amounts of data, including detailed building models. These transfers can involve thousands of files, leading to high data volumes and traffic spikes. On the other hand, certain client interactions, like user information requests, require significantly less data. Therefore, a networking solution capable of efficiently managing varied communication volumes is essential.

Another critical aspect is the coexistence of both client-to-service and service-to-service communications within the infrastructure. Client-to-service interactions are crucial for retrieving information from the cloud to the client. However, a significant portion of traffic involves service-to-service communication, where client requests are routed through multiple microservices to generate a response.

The nature of traffic within Vertex Systems necessitates a networking solution that can adeptly manage and monitor both interaction types. Such a solution should facilitate seamless inter-service communication while efficiently handling external requests entering and exiting the cluster.

Given these technical considerations, the networking solution for Vertex Systems should:

1. **Support Dynamic Load Management** To accommodate varying data volumes, the solution must dynamically manage and allocate resources to handle traffic efficiently.

2. **Optimize for Mixed Traffic Patterns** The solution should effectively manage both client-to-service and service-to-service communication, offering tools for efficient routing, service discovery, and traffic management.

3. **Ensure Robust Performance and Reliability** Given the critical nature of the data handled, the solution must uphold high performance and reliability standards, even under heavy traffic.

With these business and technical requirements in mind for Vertex Systems' infrastructure, the next step is to recap the current infrastructure and understand its constraints before concluding the preferred choice for the tools.

5.4.2 Assessing Infrastructure for Networking Solutions

When selecting the appropriate networking solution for an organization, it is crucial to consider the existing infrastructure. As highlighted in the introduction of this thesis, Vertex System is deeply committed to a cloud-native architecture. This commitment is demonstrated through the use of a microservice architecture and the incorporation of cloud-native tools and technologies. Understanding the organization's infrastructure is necessary due to its impact on the networking solution.

At the heart of Vertex Systems' infrastructure lies its microservice architecture. A prime example is Vertex Sync, a new cloud product from Vertex, which consists of approximately ten distinct microservices that interact with one another. This microservice-based framework is also evident in other Vertex cloud products, such as MyVertex, which manages customer licensing information.

Vertex operates across various geographical locations, each hosting its own dedicated Kubernetes cluster for different Vertex products. This multi-cluster arrangement is a critical aspect to consider in the networking solution, highlighting the complexity and breadth of Vertex Systems' service management. Moreover, communication between different products within the Vertex Systems ecosystem often requires cross-cluster interaction, as observed in the communication between Vertex Sync and MyVertex.

Given the infrastructure landscape of Vertex Systems, it is crucial for the networking solution to enable efficient multi-cluster communication. With significant inter-cluster traffic among Vertex products, finding a networking solution that can manage effective service-to-service communication across different clusters is essential.

5.4.3 Addressing Limitations and Improving Vertex's Infrastructure

The current infrastructure of Vertex Systems, while functional, is not optimal and has room for improvement. This section discusses the limitations and potential enhancements to consider when selecting the right networking solution.

Firstly, Vertex Systems' cloud products are managed by a relatively small team of developers, and the simplicity of the infrastructure reflects this. Therefore, any new networking solution should be easy to implement and maintain, avoiding complex setups and systems that demand excessive resources from the developer teams. The ideal choice would be a solution that integrates seamlessly into the existing workflow, allowing development teams to manage the network without significant additional overhead.

Currently, the reliance on service logs makes debugging and monitoring microservices in Vertex System's cloud products time-consuming. This becomes particularly challenging when tracking requests across multiple services and in production environments with

constant service traffic. Improving infrastructure in this area involves providing developers with tools for increased observability and traceability. The system should offer clear visibility into the journey of requests across services, including those not visible to end users (service-to-service requests). Features like real-time monitoring, easy access to logs, and visual representation of traffic flow would significantly benefit the system.

Security is another critical aspect requiring enhancement in Vertex Systems' infrastructure. Presently, all endpoints of Vertex Sync are externally accessible, which may not always be preferable or secure. A more controlled and secure communication protocol between different services is necessary. The networking solution should incorporate robust security features, aligning with zero-trust security principles. This includes ensuring secure communication and authentication between services and exposing endpoints only when necessary.

By addressing these specific concerns, the chosen solution can not only resolve existing challenges but also significantly enhance the overall efficiency and security of Vertex Systems' network infrastructure.

5.4.4 Recommending Architectural Solutions for Vertex Systems

Given the detailed analysis of Vertex Systems' business and technical requirements for the solution, infrastructure, and areas for improvement, the chosen solution should align with these criteria. As emphasized throughout this thesis, the focus here is on architectural approaches rather than specific implementations, providing Vertex Systems with a strategic direction for their networking infrastructure.

Firstly, the solution should ensure robust performance and reliability in the system. When evaluating the options, both the API gateway and service mesh can address these requirements. However, considering Vertex Systems' need for cross-cluster communication, the service mesh is more preferable in terms of system performance due to its ability to handle requests without requiring extra network hops.

Secondly, the system should incorporate the enhanced observability and traceability features that Vertex Systems requires. In this aspect, a service mesh again proves advantageous by providing visibility into east-west traffic. This capability facilitates easier monitoring and debugging by offering real-time metrics, logging, and tracing capabilities to the system.

The third point in favor of a service mesh is its ability to address security and zero-trust principles. With considerable traffic occurring between services, it is better to ensure security through a service mesh, which can achieve this without adding additional network hops. A service mesh that supports mutual TLS (Transport Layer Security)

for encrypted communication aligns with these goals by ensuring secure and protected communication of sensitive data.

While the implementation of a service mesh is crucial, there also needs to be a mechanism to handle north/south traffic in the system, ensuring that traffic successfully reaches the services. A combination of an API gateway and a service mesh can effectively address both client-to-service and service-to-service communication. In this context, whether north/south traffic is managed with an Ingress controller or an API gateway is not a primary concern due to the plan to integrate API management capabilities at a later stage. This means that the current north/south traffic management will be re-implemented in the future. However, by introducing a simple north/south traffic solution to the system now, we can effectively manage external traffic and provide features like rate limiting and endpoint security.

Considering the unique requirements of Vertex Systems, a strategic architectural choice involves a combination of an API gateway to manage external traffic and a service mesh for internal traffic. This dual approach effectively addresses the varied traffic patterns and the need for enhanced security, performance, and reliability within Vertex Systems' infrastructure.

While this section does not recommend specific products, Vertex Systems should take into account the following factors when implementing these recommendations:

1. **Vendor Compatibility** The chosen networking solutions should be compatible with each other and integrate seamlessly. Utilizing the Gateway API in both solutions would be beneficial from a developer's perspective.

2. **Scalability and Flexibility** The solution must be scalable and flexible enough to adapt to the evolving needs of the system.

3. **Ease of Use and Maintenance** Considering the limited resources, the solution should be developer-friendly and require minimal maintenance.

Refined networking solution recommendations for Vertex Systems

Given these considerations for the system, we can further refine the selection of an appropriate solution within the service mesh landscape, based on the discussion presented in the thesis. This refined analysis leads to the exclusion of certain solutions and their architectures, aligning with the specific requirements of the system.

Firstly, Vertex Sync and other Vertex Systems cloud products need to be cloud-agnostic, accommodating Vertex Systems' operation in a global context that may require deployment across multiple cloud providers. Therefore, cloud provider-specific service

meshes, such as Google's Apigee or AWS's App Mesh, are not suitable due to their limited portability outside their respective cloud infrastructures. To avoid vendor lock-in and ensure adaptability, a service mesh that operates independently of any specific cloud provider's infrastructure is preferred.

Additionally, the microservice architecture of Vertex Systems necessitates support for multi-cluster environments in the service mesh. This is crucial due to the decision to host different Vertex products in separate clusters. While some service meshes manage cross-cluster communication through an API gateway or Ingress controller, others lack a mechanism for efficient cross-cluster communication. This would be suboptimal for Vertex Systems. Therefore, the ideal service mesh should facilitate direct and efficient cross-cluster communication without relying on external components.

Finally, given Vertex Systems' relatively small development teams and the need for manageable infrastructure complexity, the service mesh must be developer-friendly. Although Envoy-based service meshes like Istio offer a wide range of features, their complexity, especially in sidecar implementation, may not be practical in this context. Ambient mesh variations of Envoy present a simpler alternative, but the priority should be on service meshes that are designed for simplicity and ease of use.

One of the most distinct service meshes introduced in the thesis was Cilium. It emerges as a notable option, particularly due to its optimizations in high-traffic scenarios. Cilium also offers a sidecarless mode, which improves efficiency by reducing the number of necessary components. However, this architectural choice introduces additional operational and security complexities, which could be challenging for teams with limited resources. The key consideration at this point is balancing immediate needs with future scalability. While Cilium's advanced features and performance benefits are attractive, the immediate focus should be on networking solutions that cater to the current needs of Vertex Systems. However, there is potential for future integration of Cilium into the infrastructure, owing to its strengths in interoperability with other service meshes and control planes. This makes Cilium a viable candidate for later phases of Vertex Systems' networking strategy.

5.4.5 Conclusion and Recommendations for Vertex Systems

As this thesis concludes, the discussions presented provide Vertex Systems with a framework for selecting a networking solution that aligns with its specific requirements and infrastructure. Based on a comprehensive evaluation of Kubernetes networking solutions and considering Vertex Systems' context, a combination of an API gateway and a service mesh is the preferred solution for the system. Specifically, there is an emphasis on adopting a service mesh with either a sidecarless architecture or proprietary proxy technology, considering their potential benefits and alignment with Vertex Systems'

infrastructure constraints.

The ultimate decision of the implementation is not covered in this thesis, but recommendations are provided to evaluate the best possible implementation for the system. The discussion highlights two potential service mesh approaches for Vertex Systems:

1. **Service Mesh with Sidecarless Architecture** This architecture could be beneficial for Vertex Systems due to its simplicity and reduced operational overhead. Utilizing a sidecarless service mesh, Vertex Systems can manage all necessary service-to-service communication features with less impact on system resources and reduced overall complexity.

2. **Service Mesh with Proprietary Proxy Technology** Alternatively, a service mesh utilizing its own proxy technology could also be a suitable solution. This approach could further increase performance benefits and optimize complexity, allowing for more controlled and efficient traffic management, which is crucial given Vertex Systems' traffic volume.

For external traffic management, an API gateway remains essential. Consequently, some form of API gateway or Ingress controller that integrates well with the chosen service mesh should be implemented.

While the current recommendations lean towards simpler and more manageable solutions, the scalability of the system should also be considered. As Vertex Systems grows, the networking infrastructure may require more performant solutions, potentially making solutions like Cilium relevant in the future.

The proposed combination of an API gateway and a service mesh, particularly one with a sidecarless architecture or proprietary proxy technology, offers Vertex Systems an effective balance between performance, manageability, and future scalability. This strategic recommendation is designed to meet the current needs of Vertex Systems while also providing the flexibility to adapt and evolve with the company's growth. By focusing on these architectural approaches, Vertex Systems can optimize its network infrastructure for efficiency, security, and ease of use, ensuring robust support for its dynamic cloud-native environment.

5.5 Addressing Research Limitations and Future Directions

This thesis provides an in-depth exploration of Kubernetes networking solutions, focusing particularly on API gateways and service meshes in a Cloud Native context. However, it is important to acknowledge certain limitations.

Firstly, the research predominantly concentrates on Kubernetes as the primary container orchestration platform within the cloud-native landscape. While Kubernetes is indeed the dominant platform for container orchestration, it's essential to recognize that cloud-native architecture does not inherently require Kubernetes as the orchestration platform. There are alternatives to Kubernetes that can offer similar networking approaches in the cloud-native environments. Nevertheless, due to its wide adoption and industry support, this research focused solely on solutions that integrate with the Kubernetes ecosystem. The implementations discussed in this thesis may not possess the same capabilities outside of Kubernetes environments. Therefore, if an organization wishes to explore alternatives to Kubernetes, their options for networking solutions might be limited.

Another limitation of this thesis is that it primarily focuses on API gateways and service meshes operating within Kubernetes. It is important to acknowledge, however, that API gateways can also function outside of the Kubernetes cluster. Some API gateways are specifically designed to operate externally, managing traffic for services that may not be within the Kubernetes environment. This research does not explore scenarios where the API gateway infrastructure is managed outside of Kubernetes. Nonetheless, there are fully managed API gateway solutions available outside of Kubernetes, such as Zuplo (Zuplo, 2024). Consequently, the findings and comparisons presented in this thesis might not completely encompass all the possibilities in cloud-native API management, especially for configurations external to Kubernetes.

Finally, this thesis adopts an approach of comparing Kubernetes networking solutions from a broad architectural standpoint. Specific implementations are mentioned to illustrate various architectural models, but a detailed comparative analysis of these implementations is not within this research's scope. The thesis provides an overview of different types of architectures and their general functionalities, rather than an in-depth examination of the differences between various API gateway and service mesh providers. For organizations looking to adopt a specific solution, a more detailed comparison of particular implementations would be beneficial. This research aids in understanding the broader array of choices upon which the implementations of these solutions are based.

In summary, while this research offers valuable insights into Kubernetes networking, particularly regarding API gateways and service meshes in cloud-native architecture, its discussion is confined to Kubernetes solutions from a broad architectural perspective.

The findings and discussions presented in this thesis highlight several areas for potential future research. Two main areas for future research, based on the limitations the thesis found, are studying how API gateways are deployed and doing a more detailed comparison of different API gateway implementations.

There is a notable gap in current research regarding the various deployment models

for API gateways. These gateways do not necessarily have to be deployed within a cluster, presenting multiple options for organizations. Future research could delve into the comparative advantages and disadvantages of situating API gateways inside versus outside the cluster. This exploration could extend to various deployment scenarios, including considerations of Ingress Controllers and service meshes, to determine the optimal placement for API gateways. Moreover, scenarios where both API gateways and service meshes are used concurrently, though common, have not been extensively researched. Investigating this area could provide valuable guidelines for implementing these solutions in real-world systems.

While the thesis discussed networking solutions from an architectural perspective, a detailed comparative analysis of specific implementations was beyond its scope. Future research could focus on a more detailed comparison of individual networking solutions, emphasizing critical metrics relevant to an organization's needs. These metrics might include performance indicators for routing and resource consumption, such as CPU and memory usage, for each implementation. Conducting such an analysis would offer practical insights into the advantages and disadvantages between implementations, thereby aiding organizations in making more informed decisions based on data. This research focused on a broader range of implementations, comparing them across various dimensions to provide a comprehensive understanding of their underlying strengths and weaknesses.

5.6 Conclusions

This thesis has provided a comprehensive analysis of cloud-native networking, focusing on the evolution of tools such as API gateways, introducing various service networking approaches in cloud-native architectures, and showcasing the relative strengths and weaknesses of these networking solutions. These findings offer valuable insights for organizations seeking to address their networking challenges in Kubernetes environments. This conclusion summarizes the key findings and recommendations, acknowledging their implications for the broader cloud-native landscape and specifically for organizations looking to utilize the tools discussed.

Firstly, the research highlighted significant advancements in API gateways, demonstrating a shift from integrating API management functionalities in API gateways to unbundling API management for more commoditized, cloud-native solutions. This evolution reflects a broader trend towards more modular and flexible solutions, facilitating effective communication for north/south traffic.

Additionally, the thesis explored diverse traffic management approaches within Kubernetes networking, emphasizing the distinction and convergence in features between API

gateways, Ingress controllers, and service meshes. This analysis underscored the importance of selecting the right solution based on an organization's specific traffic flow.

The discussion also delved into the strengths and weaknesses of various Kubernetes networking solutions, illuminating how different solutions and architectures impact the characteristics of the network. The comparative strengths of API gateways in client-to-service communication and the advantages of service meshes in service-to-service communication was explored in detail.

These findings assist organizations in adopting the right networking solution for their infrastructure. The thesis emphasized the importance of selecting a Kubernetes networking solution that aligns with specific organizational needs.

For Vertex Systems, the thesis recommends a combination of an API gateway and a service mesh, tailored to the unique requirements of their infrastructure. The focus was on sidecarless architecture and proprietary proxy technology in service meshes. This choice aims to balance performance and manageability for Vertex Systems.

This research contributes to the understanding of Kubernetes networking in cloud-native environments. It aids organizations in making informed decisions about their networking infrastructure, ensuring that their chosen solutions align with their specific needs and goals. The findings of the thesis not only highlight the current state of networking but also showcase the evolving trends of these solutions.

References

Adinegoro, F., Rahmania, C., Zaini, I. N., Negara, R. M., and Hertiana, S. N. (2022). Latency and ram usage comparison of advanced and lightweight service mesh. In *2022 5th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*, pages 369–372. IEEE.

Ali, S. A. and Zafar, M. W. (2022). Api gateway architecture explained. *INTERNATIONAL JOURNAL OF COMPUTER SCIENCE AND TECHNOLOGY*, 6(4):54–98.

Ambassador Labs (2023a). Edge stack. Availbale at: `https://www.getambassador.io/products/edge-stack/api-gateway`. Referenced: 13.11.2023.

Ambassador Labs (2023b). Edge stack kubernetes-native api gateway. Availbale at: `https://www.getambassador.io/products/edge-stack/api-gateway`. Referenced: 22.11.2023.

Balalaie, A., Heydarnoori, A., and Jamshidi, P. (2016). Microservices architecture enables devops: Migration to a cloud-native architecture. *Ieee Software*, 33(3):42–52.

Bernstein, D. (2014). Containers and cloud: From lxc to docker to kubernetes. *IEEE cloud computing*, 1(3):81–84.

Brajesh, D. (2017). Api management—an architect's guide to developing and managing apis for your organization. *Bangalore, India: APress*.

Cloud Native Computing Foundation (2022). Cncf annual survey 2022. Availbale at: `https://www.cncf.io/reports/cncf-annual-survey-2022/#findings`. Referenced: 18.10.2023.

Cloud Native Computing Foundation (2023a). Cloud native computing foundation: Cloud native definition. Availbale at: `https://www.cncf.io/about/who-we-are/`. Referenced: 16.10.2023.

Cloud Native Computing Foundation (2023b). Graduated and incubating projects. Availbale at: `https://www.cncf.io/projects/`. Referenced: 10.11.2023.

Davis, C. (2019). *Cloud Native Patterns: Designing Change-Tolerant Software*. Simon and Schuster.

De, B. and De, B. (2017). *API management*. Springer.

Duarte Maia, J. T. and Figueiredo Correia, F. (2022). Service mesh patterns. In *Proceedings of the 27th European Conference on Pattern Languages of Programs*, pages 1–12.

Envoy (2023a). Goals | envoy gateway. Availbale at: https://gateway.envoyproxy.io/v0.6.0/design/goals/. Referenced: 13.11.2023.

Envoy (2023b). System design | envoy gateway. Availbale at: https://gateway.envoyproxy.io/v0.6.0/design/system-design/. Referenced: 13.11.2023.

Farnham, T. (2023). Supporting disconnected operation of stateful services using an envoy enabled dynamic microservices approach. In *CLOSER*, pages 115–122.

Gamez-Diaz, A., Fernandez, P., and Ruiz-Cortés, A. (2019). Governify for apis: Sla-driven ecosystem for api governance. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1120–1123.

Gannon, D., Barga, R., and Sundaresan, N. (2017). Cloud-native applications. *IEEE Cloud Computing*, 4(5):16–21.

Graf, T. (2022). Cilium service mesh – everything you need to know. Availbale at: https://isovalent.com/blog/post/cilium-service-mesh/. Referenced: 16.11.2023.

Gunatilaka, P. (2023). The future of api gateways on kubernetes. Availbale at: https://www.cncf.io/blog/2023/08/14/the-future-of-api-gateways-on-kubernetes/. Referenced: 18.10.2023.

Harms, H., Rogowski, C., and Lo Iacono, L. (2017). Guidelines for adopting frontend architectures and patterns in microservices-based systems. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, pages 902–907.

Howard, J., Jackson, E. J., Kohavi, Y., Levine, I., Pettit, J., and Sun, L. (2023). Istio / ambient mesh. Availbale at: https://istio.io/v1.15/blog/2022/introducing-ambient-mesh/. Referenced: 15.11.2023.

Istio (2023). Istio. Availbale at: https://istio.io/. Referenced: 13.11.2023.

Istio (2023). Istio / ambient mesh. Availbale at: https://istio.io/latest/docs/ops/ambient/. Referenced: 15.11.2023.

James Gough, Daniel Bryant, M. A. (2020). *Mastering API Architecture*. O'Reilly Media.

Klein, M. and Reisz, W. (2022). Matt klein on envoy gateway. Availbale at: https://www.infoq.com/podcasts/matt-klein-envoy-gateway/. Referenced: 13.11.2023.

KrakenD (2016). Api gateway benchmarking with krakend. Availbale at: https://www.krakend.io/docs/benchmarks/api-gateway-benchmark/. Referenced: 23.11.2023.

Kratzke, N. and Quint, P.-C. (2017). Understanding cloud-native applications after 10 years of cloud computing-a systematic mapping study. *Journal of Systems and Software*, 126:1–16.

Kubernetes (2023a). Kubernetes: Cluster architecture. Availbale at: `https://kubernetes.io/docs/concepts/architecture/`. Referenced: 17.10.2023.

Kubernetes (2023b). Kubernetes: Glossary. Availbale at: `https://kubernetes.io/docs/reference/glossary/?fundamental=true`. Referenced: 17.10.2023.

Kubernetes (2023c). Kubernetes: Ingress. Availbale at: `https://kubernetes.io/docs/concepts/services-networking/ingress/`. Referenced: 27.10.2023.

Kubernetes (2023d). Kubernetes: Production-grade container orchestration. Availbale at: `https://kubernetes.io/`. Referenced: 17.10.2023.

Kubernetes Gateway API (2023a). Gamma - kubernetes gateway api. Availbale at: `https://gateway-api.sigs.k8s.io/concepts/gamma/`. Referenced: 9.11.2023.

Kubernetes Gateway API (2023b). Implementations - kubernetes gateway api. Availbale at: `https://gateway-api.sigs.k8s.io/implementations/`. Referenced: 27.10.2023.

Kubernetes Gateway API (2023c). Implementer's guide - kubernetes gateway api. Availbale at: `https://gateway-api.sigs.k8s.io/reference/implementers-guide/`. Referenced: 08.11.2023.

Kubernetes Gateway API (2023d). Introduction - kubernetes gateway api. Availbale at: `https://gateway-api.sigs.k8s.io/`. Referenced: 27.10.2023.

Kubernetes Gateway API (2023e). Use cases - kubernetes gateway api. Availbale at: `https://gateway-api.sigs.k8s.io/concepts/use-cases/`. Referenced: 27.10.2023.

Li, R. (2020). The future of cloud-native api gateways. Availbale at: `https://www.infoq.com/presentations/api-gateways-history/`. Referenced: 18.10.2023.

Li, W., Lemieux, Y., Gao, J., Zhao, Z., and Han, Y. (2019). Service mesh: Challenges, state of the art, and future research opportunities. In *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 122–1225. IEEE.

Linkerd (2023a). linkerd2. `https://github.com/linkerd/linkerd2`. Commit hash: 83cab30.

Linkerd (2023b). linkerd2-proxy. `https://github.com/linkerd/linkerd2-proxy`. Commit hash: 048542a.

Linkerd (2023). Overview | linkerd. Availbale at: https://linkerd.io/2.14/overview/. Referenced: 14.11.2023.

Luksa, M. (2017). *Kubernetes in action*. Simon and Schuster.

Mathijssen, M., Overeem, M., and Jansen, S. (2020). Identification of practices and capabilities in api management: a systematic literature review. *arXiv preprint arXiv:2006.10481*.

Miyachi, C. (2021). The rise of kubernetes. In *2021 Cloud Continuum*, pages 1–5. IEEE Computer Society.

Montesi, F. and Weber, J. (2016). Circuit breakers, discovery, and api gateways in microservices. *arXiv preprint arXiv:1609.05830*.

Morgan, W. (2023). Kubernetes 1.28: Revenge of the sidecars? Availbale at: https://buoyant.io/blog/kubernetes-1-28-revenge-of-the-sidecars. Referenced: 10.11.2023.

Mulesoft (2019). How to design and manage APIs. White paper, Mulesoft.

Neal, T., Bertschy, M., Kanzhelev, S., Kim, G., and Kularathna, S. (2023). Kubernetes v1.28: Introducing native sidecar containers. Availbale at: https://kubernetes.io/blog/2023/08/25/native-sidecar-containers/. Referenced: 10.11.2023.

Neumann, A., Laranjeiro, N., and Bernardino, J. (2018). An analysis of public rest web service apis. *IEEE Transactions on Services Computing*, 14(4):957–970.

OpenID Foundation (2023). Openid. Availbale at: https://openid.net/. Referenced: 05.12.2023.

Palladino, M. (2023). The difference between api gateways and service mesh. Availbale at: https://konghq.com/resources/e-book/api-gateways-vs-service-mesh. Referenced: 26.10.2023.

Posta, C. (2020). Can you replace api management with service mesh? Availbale at: https://www.solo.io/blog/webinar-recap-can-you-replace-api-management-with-service-mesh/. Referenced: 26.10.2023.

Posta, C. (2023). Full lifecycle api management is dead. In "Software Integration" report.

Proxy, E. (2023a). Envoy proxy - community. Availbale at: https://www.envoyproxy.io/community. Referenced: 10.11.2023.

Proxy, E. (2023b). Envoy proxy - what is envoy. Availbale at: https://www.envoyproxy.io/docs/envoy/v1.28.0/intro/what_is_envoy. Referenced: 10.11.2023.

Reisz, W. (2022). API Gateways & Service Mesh: What's best for us. XConf North America.

Richardson, C. (2018). *Microservices patterns: with examples in Java*. Simon and Schuster.

Saara Inc. (2023). Enroute ingress api gateway. Availbale at: https://www.getenroute.io/. Referenced: 22.11.2023.

Schermann, G., Cito, J., and Leitner, P. (2016). All the services large and micro: Revisiting industrial practice in services computing. In *Service-Oriented Computing–ICSOC 2015 Workshops: WESOA, RMSOC, ISC, DISCO, WESE, BSCI, FORMOVES, Goa, India, November 16-19, 2015, Revised Selected Papers 13*, pages 36–47. Springer.

Sedghpour, M. R. S. and Townend, P. (2022). Service mesh and ebpf-powered microservices: A survey and future directions. In *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 176–184. IEEE.

Solo.io (2020). Introducing the first developer portal for istio. Availbale at: https://www.solo.io/blog/introducing-the-first-developer-portal-for-istio/. Referenced: 30.11.2023.

Solo.io (2023). Gloo gateway. Availbale at: https://www.solo.io/products/gloo-gateway/. Referenced: 22.11.2023.

Sun, L. and Posta, C. (2022). *Istio Ambient Explained*. O'Reilly Media.

Tan, W., Fan, Y., Ghoneim, A., Hossain, M. A., and Dustdar, S. (2016). From the service-oriented architecture to the web api economy. *IEEE Internet Computing*, 20(4):64–68.

Vieira, M. A., Castanho, M. S., Pacífico, R. D., Santos, E. R., Júnior, E. P. C., and Vieira, L. F. (2020). Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)*, 53(1):1–36.

Zhao, J., Jing, S., and Jiang, L. (2018). Management of api gateway based on microservice architecture. In *Journal of Physics: Conference Series*, volume 1087, page 032032. IOP Publishing.

Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X. K., Duan, X., He, P., Krishnamurthy, A., Lentz, M., et al. (2023). Dissecting overheads of service mesh

sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, pages 142–157.

Zuplo (2024). Zuplo api management. Availbale at: `https://zuplo.com/`. Referenced: 15.01.2023.